LeanVM

draft 0.9.0 (Work In Progress)

Contents

1 Introduction

Ethereum's cryptography currently relies on elliptic curves, which are known to be vulnerable to quantum computers [1].

LeanVM is a verifiable Virtual Machine (also called zkVM for "zero knowledge VM"). It uses a minimal ISA (Instruction Set Architecture) inspired by Cairo [2].

- It's hash-based (WHIR [3]): plausibly post-quantum, conceptually simple
- It's multilinear: the trace is committed as a single multilinear polynomial (reducing proof size), and constraints are proven via sumcheck [4]
- It uses the Poseidon [5] permutation as a snark-friendly hashing primitive

2 Use-cases

LeanVM is optimized to provide a post-quantum alternative for each of the three layers of Ethereum:

2.1 Consensus Layer

- **Currently (pre-quantum):** BLS [6]: lightweight signatures of 96 bytes, with native support for aggregation.
- **With LeanVM (post-quantum):** Replace BLS by XMSS over Poseidon [7] (leanXMSS). Aggregate those signatures using leanVM. Already aggregated signatures can be further merged, via recursion. XMSS is a stateful signature scheme, but Ethereum's slot can be used as a nonce (double signing is already prevented).

See Section 7 for more details.

2.2 Execution Layer

- **Currently (pre-quantum):** ECDSA [8]
- **With LeanVM (post-quantum):** Replace ECDSA by a variant of SPHINCS+ [9] over poseidon (leanSPHINCS). Aggregate all the signatures from one block using leanVM.

See Section 7 for more details.

2.3 Data Availability Layer

- **Currently (pre-quantum):** KZG polynomial commitment scheme [10]
- **With LeanVM (post-quantum):** Reed-Solomon encode and Merkle-commit the codeword, for each blob. Prove the claimed Merkle Root was correctly computed using leanVM.

3 Preliminaries

3.1 Notations

- $[a, b] = \{a, a+1, \dots, b\}$
- $[a, b) = \{a, a+1, \dots, b-1\}$
- $\hat{eq}\big((x_1, \dots, x_n), (y_1, \dots, y_n)\big) = \prod_{i=1}^{n} \big(x_i y_i + (1 - x_i)(1 - y_i)\big)$
- $\mathsf{embed}(x_1, x_2, x_3, x_4, x_5)$ is the extension field element built from d base field elements: $x_1 + x_2 X + x_3 X^2 + x_4 X^3 + x_5 X^4$, using $X^5 + X^2 - 1$ as the irreducible polynomial defining the extension (see Section 4.1).

- Given a number of bits $n > 0$, and an n-bit integer x, we denote by $[x]^n_{\text{bits}}$ its big-endian bit-decomposition in n boolean values. For example, $[14]^4_{\text{bits}} = (1, 1, 1, 0)$
- h_{MEMORY} is the log-size of the (read-only) memory; the memory has $2^{h_{\text{MEMORY}}}$ words (always a power of 2). h_{BYTECODE} is the bytecode log-size (the bytecode has $2^{h_{\text{BYTECODE}}}$ instructions), and h_{EXEC}, h_{POSEIDON}, $h_{\text{EXTENSION}}$ are the log-heights of the three tables (see Section 5.2).
- $\mathbf{m}[i]$ is the value of the memory at index i. It is defined only for $0 \leq i < 2^{h_{\text{MEMORY}}}$; any out-of-bound access ($i \geq 2^{h_{\text{MEMORY}}}$) makes the execution *invalid*. More generally, any vector of field elements v is indexed starting at $v[0], v[1], \ldots$
- $\mathbf{m}[i..j]$ is the memory slice $(\mathbf{m}[i], \mathbf{m}[i+1], \ldots, \mathbf{m}[j-1])$.
- $||$ denotes concatenation of slices.
- 0_k denotes the all-zero vector of length k, for $k \geq 0$.
- $x \xleftarrow{\$} S$ means x is sampled uniformly at random from the finite set S.
- $[P]$ denotes the *Iverson bracket* of a proposition P: $[P] = 1$ if P holds, and 0 otherwise.

3.2 Definitions

3.2.1 Multilinear Extension (MLE)

For a vector of field elements $v \in \mathbb{F}^{2^n}$ (for some $n \geq 0$), the Multilinear Extension (MLE) of v, denoted by $\hat{v}$, is the unique multilinear polynomial in $\mathbb{F}[X_1, \ldots, X_n]$, in n variables, such that: $\forall i \in [0, 2^n), v[i] = \hat{v}([i]^n_{\text{bits}})$

Throughout, v denotes a vector and $\hat{v}$ its MLE.

3.2.2 $\mathbb{F}_p$-valued multiset

An *$\mathbb{F}_p$-valued multiset* over a set X is a function $M : X \to \mathbb{F}_p$ assigning a multiplicity in $\mathbb{F}_p$ to each element of X. Its *support* is $\text{supp}(M) = \{x \in X : M(x) \neq 0\}$, and M is the *zero multiset* when $\text{supp}(M) = \emptyset$.

3.2.3 Inner Product Commitment Scheme

An *Inner Product Commitment Scheme* is an interactive protocol where a prover commits to a vector of field elements $v \in \mathbb{F}_p^{2^n}$ (for some $n \geq 0$), and later convinces a verifier of the evaluation of $\sum_{i=0}^{2^n-1} v[i] \cdot x[i]$ for some arbitrary vector $x \in \mathbb{F}_q^{2^n}$, assuming the verifier is able to evaluate the MLE of x.

WHIR [3] is an example of Inner Product Commitment Scheme.

3.2.4 Shifted polynomial

Given a vector $v \in \mathbb{F}_p^{2^n}$, we define $\mathsf{shift}(v) \in \mathbb{F}_p^{2^n}$ by $\mathsf{shift}(v)[2^n - 1] = v[2^n - 1]$ and:

$$\forall i \in [0, 2^n - 1), \mathsf{shift}(v)[i] = v[i+1]$$

3.2.5 The next multilinear

For $n \geq 0$, we define $\hat{\mathsf{next}}$ as the multilinear polynomial in $2n$ variables whose values on the boolean hypercube are

$$\hat{\mathsf{next}}([x]^n_{\text{bits}} \,||\, [y]^n_{\text{bits}}) = \begin{cases} 1 & \text{if } y = x + 1, \\ 1 & \text{if } x = y = 2^n - 1, \\ 0 & \text{otherwise,} \end{cases} \qquad x, y \in [0, 2^n)$$

$\hat{\mathsf{next}}$ can be efficiently evaluated:

$$\hat{\mathsf{next}}((x_0,\ldots,x_{n-1}),(y_0,\ldots,y_{n-1})) \;=\; \prod_{j=0}^{n-1} x_j\, y_j \;+\; \sum_{k=0}^{n-1}\left(\prod_{j<k} \hat{eq}(x_j,y_j)\right)(1-x_k)\, y_k \left(\prod_{j>k} x_j\,(1-y_j)\right).$$

The leading product is the wrap-around case $x = y = 2^n - 1$. The k-th term of the sum is the case $y = x + 1$ with the carry stopping at bit k: the high bits ($j < k$) are unchanged, bit k flips $0 \to 1$, and the low bits ($j > k$) flip $1 \to 0$.

This enables sumcheck-based evaluation of shifted MLEs (section 3.2.4): for every $v \in \mathbb{F}_p^{2^n}$,

$$\hat{\mathsf{shift}}(v)(x) \;=\; \sum_{y\in\{0,1\}^n} \hat{\mathsf{next}}(x,y)\;\hat{v}(y),$$

3.3 Lemmas

3.3.1 Schwartz-Zippel

Lemma 1 (Schwartz-Zippel)**.** Let $P \in \mathbb{F}_q[X_1,\ldots,X_n]$ be a non-zero polynomial of total degree at most d. Then

$$\Pr_{\vec{\beta}\xleftarrow{\$}\mathbb{F}_q^n}\left[P(\vec{\beta}) = 0\right] \;\le\; \frac{d}{q}.$$

Proof. By induction on n. For $n = 1$, P is a non-zero univariate polynomial of degree d, hence has at most d roots in $\mathbb{F}_q$, so $\Pr_\beta[P(\beta) = 0] \le d/q$.

For $n \ge 2$, write P as a polynomial in X_n with coefficients in $\mathbb{F}_q[X_1,\ldots,X_{n-1}]$:

$$P(X_1,\ldots,X_n) \;=\; \sum_{k=0}^{d} X_n^k\, Q_k(X_1,\ldots,X_{n-1}),$$

and let $k^\star = \max\{k : Q_k \neq 0\}$. Then $Q_{k^\star}$ is a non-zero polynomial in $n-1$ variables of total degree at most $d - k^\star$. By the induction hypothesis, $\Pr_{\vec{\beta}_{<n}}[Q_{k^\star}(\vec{\beta}_{<n}) = 0] \le (d-k^\star)/q$. Conditioned on $Q_{k^\star}(\vec{\beta}_{<n}) \neq 0$, the polynomial $X_n \mapsto P(\vec{\beta}_{<n}, X_n)$ has degree exactly $k^\star$, hence at most $k^\star$ roots in $\mathbb{F}_q$, so $\Pr_{\beta_n}[P(\vec{\beta}) = 0 \mid Q_{k^\star}(\vec{\beta}_{<n}) \neq 0] \le k^\star/q$. A union bound gives $\Pr_{\vec{\beta}}[P(\vec{\beta}) = 0] \le d/q$. □

A useful consequence: a non-zero multilinear polynomial in n variables has total degree $\le n$, so it vanishes at a uniform $\vec{\beta} \in \mathbb{F}_q^n$ with probability at most n/q.

3.4 Constants

Symbol	Value	Description
d	5	degree of the extension field $\mathbb{F}_q$ over $\mathbb{F}_p$
ℓ_{DIGEST}	8	hash digest size, in field elements
ℓ_{PUBLIC}	ℓ_{DIGEST}	public input length (one digest)
$h^{\min}_{\text{MEMORY}}$	16	log2 of the minimum size of the memory
$h^{\max}_{\text{MEMORY}}$	26	log2 of the maximum size of the memory
$h^{\min}$	8	log2 of the minimum height shared by all tables (and by the bytecode)
$h^{\max}_{\text{EXEC}}$	24	log2 of the maximum height of the `execution` table
$h^{\max}_{\text{POSEIDON}}$	22	log2 of the maximum height of the `poseidon` table
$h^{\max}_{\text{EXTENSION}}$	22	log2 of the maximum height of the `extension` table
bus_width	16	number of field elements in the bus (see Section 5.3)
ℓ	4	$\log_2(\text{bus\_width})$
memory_sep	1	domain separator of memory interactions
bytecode_sep	2	domain separator of bytecode interactions
$\text{flag}_{\text{SINGLE}}$	1	public-input flag for a single-message aggregation (Section 7.2)
$\text{flag}_{\text{MULTI}}$	0	public-input flag for a multi-message aggregation (Section 7.2)
$n^{execution}_{cols}$	20	number of columns in the `execution` table
f^{whir}_0	7	WHIR initial folding factor
$f^{\text{whir}}_{>0}$	5	WHIR subsequent folding factor
$\text{log\_inv\_rate}^{\min}$	1	minimum value of log_inv_rate (WHIR)
$\text{log\_inv\_rate}^{\max}$	4	maximum value of log_inv_rate

4 VM specification

4.1 Field

The base field is $\mathbb{F}_p$, with $p = 2^{31} - 2^{24} + 1$ (KoalaBear prime). The extension field is $\mathbb{F}_q = \mathbb{F}_p[X]/(X^5 + X^2 - 1)$, a degree-$d$ extension ($d = 5$) with $q = p^d$. Extension field elements are represented in the canonical basis $(1, X, X^2, X^3, X^4)$ as d-tuples of base field coordinates (cf. the embed map of Section 3.1).

4.2 Poseidon specification

We use the Poseidon [5] permutation, over a state of 16 (base) field elements:

$$\texttt{POSEIDON}^{\pi} : \mathbb{F}_p^{16} \to \mathbb{F}_p^{16}$$

The MDS matrix is circulant, with first row $[1, 3, 13, 22, 67, 2, 15, 63, 101, 1, 2, 17, 11, 1, 51, 1]$.

There are $N_{\text{full}}/2 = 4$ initial full rounds, $N_{\text{partial}} = 20$ partial rounds, and $N_{\text{full}}/2 = 4$ final full rounds.

TODO explain how round constants were generated.

Poseidon can be used in "compression" mode (feed-forward):

$$\begin{aligned} \texttt{POSEIDON}^{C} : \mathbb{F}_p^{16} &\to \mathbb{F}_p^{8} \\ x &\mapsto (\texttt{POSEIDON}^{\pi}(x) + x)[0..8] \end{aligned}$$

where $+$ represents field coordinate-wise addition, and $[0..8]$ represents the first 8 elements of the result.

4.3 Memory

We use a read-only memory (also called write-once memory). Each memory word is a base field element. The memory is a 1D array of $2^{h_{\text{MEMORY}}}$ words; the log-size h_{MEMORY} is attached to each execution

of the VM and must satisfy $h^{\min}_{\text{MEMORY}} \leq h_{\text{MEMORY}} \leq h^{\max}_{\text{MEMORY}}$, with $h^{\min}_{\text{MEMORY}} = 16$ and $h^{\max}_{\text{MEMORY}} = 26$. The first ℓ_{PUBLIC} memory cells (one hash digest) hold the "public input", which informally represents the part of the memory "controlled" by the verifier of an execution proof. If a program requires bigger input, it is enought to pass as public input its hash, and to hint (see Section 6.1) the corresponding data at runtime (whose hash will be checked against the public input).

4.4 Bytecode

The bytecode is also read-only, and also has a power of two size. It lives in a separate region than the memory (Harvard architecture). Both prover and verifier are expected to have access to the bytecode. For a bytecode of size $2^{h_{\text{BYTECODE}}}$, a valid execution should start at $\mathbf{pc} = 0$, and ends at $\mathbf{pc} = 2^{h_{\text{BYTECODE}}} - 1$. In practice, it is suggested to use for the last instruction (at index $2^{h_{\text{BYTECODE}}} - 1$) a `JUMP` (section 4.7.4) that unconditionally jumps to itself (for padding reasons within the snark). The minimal bytecode log-size is $h^{\min}$.

Well-formed bytecode. The bytecode is assumed *well-formed*: every instruction is a legal encoding, as produced by Section 5.5.2. Many constraints rely on it (Section 5.5.3), without enforcing it.

4.5 Registers

The VM uses two registers **pc** and **fp** that fully describes its state. Everything else (memory and bytecode) is read-only.

- **pc**: program counter, points to the current instruction
- **fp**: frame pointer, points to the start of the current "frame" in memory

Except for JUMP, every instruction updates them by $\mathbf{pc} \leftarrow \mathbf{pc} + 1$ and $\mathbf{fp} \leftarrow \mathbf{fp}$.

4.6 Addressing modes

Most of the operands usually come in pair (x^{addr}, x):

- one *addressing mode* $x^{\mathsf{addr}} \in \{\,\texttt{imm},\ \texttt{mem},\ \texttt{fp\_rel}\,\}$
- one *value* $x \in \mathbb{F}_p$

At runtime, it resolves to:

$$\mathsf{val}(x^{\mathsf{addr}}, x) = \begin{cases} x & \text{if } x^{\mathsf{addr}} = \texttt{imm} \quad \text{(immediate)} \\ \mathbf{m}[\mathbf{fp} + x] & \text{if } x^{\mathsf{addr}} = \texttt{mem} \quad \text{(memory)} \\ \mathbf{fp} + x & \text{if } x^{\mathsf{addr}} = \texttt{fp\_rel} \quad \text{(fp-relative)} \end{cases}$$

4.7 Instruction Set Architecture

The instruction set comprises 4 *core instructions* (ADD, MUL, DEREF, JUMP) and 2 *precompiles* (POSEIDON_OP, EXTENSION_OP).

4.7.1 ADD

An addition over the base field $\mathbb{F}_p$.

$$\texttt{ADD}[\ \alpha^{\mathsf{addr}},\ \alpha,\ \ \beta^{\mathsf{addr}},\ \beta,\ \ \gamma^{\mathsf{addr}},\ \gamma\ ]$$

Operands:

- $\alpha, \beta, \gamma \in \mathbb{F}_p$
- $\alpha^{\mathsf{addr}}, \beta^{\mathsf{addr}} \in \{\texttt{imm}, \texttt{mem}\}$
- $\gamma^{\mathsf{addr}} \in \{\texttt{imm}, \texttt{mem}, \texttt{fp\_rel}\}$

Semantics. Write $a = \mathsf{val}(\alpha^{\mathsf{addr}}, \alpha)$, $b = \mathsf{val}(\beta^{\mathsf{addr}}, \beta)$, $c = \mathsf{val}(\gamma^{\mathsf{addr}}, \gamma)$.

$$a + c = b$$

4.7.2 MUL

A multiplication over the base field $\mathbb{F}_p$.

$$\texttt{MUL}[\ \alpha^{\mathsf{addr}},\ \alpha,\ \ \beta^{\mathsf{addr}},\ \beta,\ \ \gamma^{\mathsf{addr}},\ \gamma\ ]$$

Same operands as `ADD`. Reusing the notations above, the semantics is

$$a \cdot c = b$$

4.7.3 DEREF

A pointer dereference: accesses memory at an address read from another memory cell.

$$\texttt{DEREF}[\ \alpha,\ \ \beta,\ \ \gamma^{\mathsf{addr}},\ \gamma\ ]$$

Operands:

- $\alpha, \beta, \gamma \in \mathbb{F}_p$
- $\gamma^{\mathsf{addr}} \in \{\texttt{imm}, \texttt{mem}, \texttt{fp\_rel}\}$

Semantics:

$$\mathbf{m}[\ \mathbf{m}[\mathbf{fp} + \alpha] + \beta\ ]\ =\ \mathsf{val}(\gamma^{\mathsf{addr}}, \gamma).$$

4.7.4 JUMP

A conditional jump.

$$\texttt{JUMP}[\ \alpha^{\mathsf{addr}},\ \alpha,\ \ \beta^{\mathsf{addr}},\ \beta,\ \ \gamma^{\mathsf{addr}},\ \gamma\ ]$$

Operands:

- $\alpha, \beta, \gamma \in \mathbb{F}_p$
- $\alpha^{\mathsf{addr}}, \beta^{\mathsf{addr}} \in \{\texttt{imm}, \texttt{mem}\}$
- $\gamma^{\mathsf{addr}} \in \{\texttt{imm}, \texttt{mem}, \texttt{fp\_rel}\}$

Semantics: Write $\text{cond} = \mathsf{val}(\alpha^{\mathsf{addr}}, \alpha)$, $\text{updated\_pc} = \mathsf{val}(\beta^{\mathsf{addr}}, \beta)$, and $\text{updated\_fp} = \mathsf{val}(\gamma^{\mathsf{addr}}, \gamma)$. The condition is constrained to be boolean, $\text{cond} \in \{0, 1\}$, and the registers are updated by

$$(\mathbf{pc}, \mathbf{fp}) \ \leftarrow\ \begin{cases} (\ \text{updated\_pc},\ \text{updated\_fp}\ ) & \text{if cond} = 1 \\ (\ \mathbf{pc} + 1,\ \mathbf{fp}\ ) & \text{if cond} = 0 \end{cases}$$

4.7.5 POSEIDON_OP

A precompile to compute Poseidon in permutation or compression mode.

$$\texttt{POSEIDON\_OP}[\ \alpha^{\mathsf{addr}},\ \alpha,\ \ \beta^{\mathsf{addr}},\ \beta,\ \ \gamma^{\mathsf{addr}},\ \gamma,\ \ \texttt{permute},\ \ \texttt{out8},\ \ \texttt{out4},\ \ \delta\ ]$$

Operands:

- $\alpha, \beta, \gamma \in \mathbb{F}_p$
- $\alpha^{\mathsf{addr}}, \beta^{\mathsf{addr}}, \gamma^{\mathsf{addr}} \in \{\texttt{imm}, \texttt{mem}, \texttt{fp\_rel}\}$, with $\alpha^{\mathsf{addr}} = \texttt{fp\_rel} \Leftrightarrow \beta^{\mathsf{addr}} = \texttt{fp\_rel}$
- $\texttt{permute} \in \{0, 1\}$; the pair $(\texttt{out8}, \texttt{out4})$ gives the output length, $(1, 0) \mapsto 8$, $(0, 1) \mapsto 4$, $(0, 0) \mapsto 16$ (so $\texttt{out8} \cdot \texttt{out4} = 0$); $\delta \in \mathbb{F}_p \cup \{\perp\}$. Valid combinations: compression outputs 8 or 4, permutation outputs 16 or 8 (i.e. $\texttt{permute} \cdot \texttt{out4} = 0$, and the 16-element output requires $\texttt{permute} = 1$).

Semantics: Define the pointers $a = \mathsf{val}(\alpha^{\mathsf{addr}}, \alpha)$, $b = \mathsf{val}(\beta^{\mathsf{addr}}, \beta)$, $c = \mathsf{val}(\gamma^{\mathsf{addr}}, \gamma)$.
Input. The 16-element input $x = L \,||\, R \in \mathbb{F}_p^{16}$ is built from a left half L and a right half R:

$$L = \begin{cases} \mathbf{m}[a..a+8] & \text{if } \delta = \bot \\ \mathbf{m}[\delta..\delta+4] \,||\, \mathbf{m}[a..a+4] & \text{if } \delta \neq \bot \end{cases} \qquad R = \mathbf{m}[b..b+8].$$

Output.

$$\begin{cases} \mathbf{m}[c..c+16] = \texttt{POSEIDON}^{\pi}(x) & \text{if } \texttt{permute} = 1,\ \texttt{out8} = \texttt{out4} = 0 \\ \mathbf{m}[c..c+8] = \texttt{POSEIDON}^{\pi}(x)[0..8] & \text{if } \texttt{permute} = 1,\ \texttt{out8} = 1 \\ \mathbf{m}[c..c+8] = \texttt{POSEIDON}^{C}(x) & \text{if } \texttt{permute} = 0,\ \texttt{out8} = 1 \\ \mathbf{m}[c..c+4] = \texttt{POSEIDON}^{C}(x)[0..4] & \text{if } \texttt{permute} = 0,\ \texttt{out4} = 1 \end{cases}$$

4.7.6 EXTENSION_OP

A precompile to compute various operations in the extension field, useful for recursion.

$$\texttt{EXTENSION\_OP}[\ \alpha^{\mathsf{addr}},\ \alpha,\ \ \beta^{\mathsf{addr}},\ \beta,\ \ \gamma^{\mathsf{addr}},\ \gamma,\ \ \mathsf{op},\ \ \mathsf{flag\_be},\ \ N\ ]$$

Operands:

- $\alpha, \beta, \gamma \in \mathbb{F}_p$
- $\alpha^{\mathsf{addr}}, \beta^{\mathsf{addr}}, \gamma^{\mathsf{addr}} \in \{\texttt{imm}, \texttt{mem}, \texttt{fp\_rel}\}$, with $\alpha^{\mathsf{addr}} = \texttt{fp\_rel} \Leftrightarrow \beta^{\mathsf{addr}} = \texttt{fp\_rel}$
- $\mathsf{op} \in \{\text{ADD}, \text{DOT\_PRODUCT}, \text{EQ}\}$, $\mathsf{flag\_be} \in \{0, 1\}$
- $N \geq 1$

Semantics: Define the pointers $a = \mathsf{val}(\alpha^{\mathsf{addr}}, \alpha)$, $b = \mathsf{val}(\beta^{\mathsf{addr}}, \beta)$, $c = \mathsf{val}(\gamma^{\mathsf{addr}}, \gamma)$.
Input. For each $i \in [0, N)$, define $b_i = \mathsf{embed}\big(\mathbf{m}[b + di \,..\, b + di + d]\big) \in \mathbb{F}_q$, and:

$$a_i = \begin{cases} \mathbf{m}[a+i] \in \mathbb{F}_p & \text{if } \mathsf{flag\_be} = 1 \quad \text{(base-extension)} \\ \mathsf{embed}\big(\mathbf{m}[a + di \,..\, a + d(i+1)]\big) \in \mathbb{F}_q & \text{if } \mathsf{flag\_be} = 0 \quad \text{(extension-extension)} \end{cases}$$

Output.

$$\mathsf{embed}\big(\mathbf{m}[c \,..\, c+d]\big) = \begin{cases} \sum_{i=0}^{N-1} (a_i + b_i) & \text{if } \mathsf{op} = \text{ADD} \\ \sum_{i=0}^{N-1} a_i \cdot b_i & \text{if } \mathsf{op} = \text{DOT\_PRODUCT} \\ \prod_{i=0}^{N-1} \hat{eq}(a_i, b_i) & \text{if } \mathsf{op} = \text{EQ} \end{cases}$$

4.8 Example: Fibonacci Program

The following program takes two base field elements (n, r) as public input (occupying the first two cells of the fixed $\ell_{\text{PUBLIC}} = 8$ public-input slots), and proves that the n-th Fibonacci number, reduced modulo p, equals r.

```
[program]
   0: ADD[imm, 0,  imm, 0,  mem, 2]
   1: DEREF[2,  0,  mem, 3]
   2: ADD[imm, 1,  mem, 4,  mem, 3]
   3: DEREF[5,  0,  imm, 1]
   4: DEREF[5,  1,  imm, 1]
   5: ADD[imm, 1,  mem, 3,  mem, 6]
   6: DEREF[7,  0,  imm, 34]
   7: DEREF[7,  1,  fp_rel, 0]
   8: DEREF[7,  2,  imm, 0]
   9: DEREF[7,  3,  mem, 5]
  10: DEREF[7,  4,  mem, 3]
  11: DEREF[7,  5,  mem, 6]
  12: JUMP[imm, 1,  imm, 13,  mem, 7]
  13: ADD[mem, 6,  mem, 2,  mem, 5]
  14: MUL[mem, 6,  mem, 8,  mem, 7]
  15: ADD[mem, 9,  imm, 1,  mem, 8]
  16: MUL[mem, 9,  imm, 0,  mem, 6]
  17: JUMP[mem, 8,  imm, 19,  fp_rel, 0]
  18: JUMP[imm, 1,  imm, 32,  fp_rel, 0]
  19: ADD[mem, 3,  mem, 10,  mem, 2]
  20: DEREF[10,  0,  mem, 11]
  21: DEREF[10,  1,  mem, 12]
  22: ADD[mem, 11,  mem, 13,  mem, 12]
  23: DEREF[10,  2,  mem, 13]
  24: ADD[imm, 1,  mem, 14,  mem, 2]
  25: DEREF[15,  0,  mem, 0]
  26: DEREF[15,  1,  mem, 1]
  27: DEREF[15,  2,  mem, 14]
  28: DEREF[15,  3,  mem, 3]
  29: DEREF[15,  4,  mem, 4]
  30: DEREF[15,  5,  mem, 5]
  31: JUMP[imm, 1,  imm, 13,  mem, 15]
  32: JUMP[imm, 1,  mem, 0,  mem, 1]
  33: JUMP[imm, 1,  imm, 34,  fp_rel, 0]
  34: ADD[mem, 5,  mem, 8,  mem, 6]
  35: DEREF[8,  0,  mem, 9]
  36: DEREF[2,  1,  mem, 9]
  37: ADD[imm, 0,  imm, 0,  mem, 10]
  38: JUMP[imm, 1,  imm, 255,  mem, 10]

  39: MUL[imm, 0,  imm, 1,  fp_rel, 0]
  40: MUL[imm, 0,  imm, 1,  fp_rel, 0]
      ... (padding)
 253: MUL[imm, 0,  imm, 1,  fp_rel, 0]
 254: MUL[imm, 0,  imm, 1,  fp_rel, 0]

 255: JUMP[imm, 1,  imm, 255,  fp_rel, 0]
```

5 Proving system

5.1 Stacking multilinear polynomials

5.1.1 Protocol

Given an inner product PCS (section 3.2.3), we present a protocol that lets a prover commit to many multilinear polynomials of potentially different sizes, and later prove their respective evaluations, and evaluations of their shifted MLEs (section 3.2.4), at various points.

Setup

The prover has access to $n > 0$ multilinear polynomials $\hat{P}_1, \ldots, \hat{P}_n$ with $\nu_1, \ldots, \nu_n$ variables respectively, given by their evaluation vectors $P_1, \ldots, P_n$ of respective lengths $2^{\nu_1}, \ldots, 2^{\nu_n}$. We assume without loss of generality $\nu_1 \geq \nu_2 \geq \cdots \geq \nu_n$.

We stack them into a single vector of length 2^ν, where $\nu = \lceil \log_2(\sum_i 2^{\nu_i}) \rceil$:

$$S \;=\; \mathsf{stack}[P_1, \ldots, P_n] \;=\; P_1 \,\|\, P_2 \,\|\, \ldots \,\|\, P_n \,\|\, 0_{\,2^\nu - \sum_i 2^{\nu_i}},$$

and commit to S via the inner product PCS.

Claims

There are J *claims*, indexed by $j \in [1, J]$. Each claim concerns one inner polynomial P_i and one point $z \in \mathbb{F}_q^{\nu_i}$, and is of one of two kinds:

- a *normal claim* asks for the value $\hat{P}_i(z)$;
- a *shift claim* asks for the value $\widehat{\mathsf{shift}}(P_i)(z)$.

The prover answers claim j with a *claimed value* $c_j \in \mathbb{F}_q$.

From local claims over P_i to global claims over S

Since $\nu_1 \geq \cdots \geq \nu_n$, the block of P_i inside S starts at offset $o_i = \sum_{i'<i} 2^{\nu_{i'}}$, a multiple of 2^{ν_i}. Let $\mathsf{sel}_i \in \{0,1\}^{\nu - \nu_i}$ be the big-endian encoding of $o_i / 2^{\nu_i}$, the *boolean selector* of block i; then $\hat{P}_i = \hat{S}(\mathsf{sel}_i, \cdot)$.

This rewrites every claim as a **weighted sum** over the single committed polynomial $\hat{S}$. Fix a claim j, on the inner polynomial P_i at the point z. If it is a *normal* claim, then since $\hat{P}_i = \hat{S}(\mathsf{sel}_i, \cdot)$,

$$c_j = \hat{S}(\mathsf{sel}_i, z) = \sum_{w \in \{0,1\}^\nu} \hat{eq}\big((\mathsf{sel}_i, z),\, w\big)\ \hat{S}(w).$$

If it is a *shift* claim, the $\hat{\mathsf{next}}$ identity of section 3.2.5 and $\hat{P}_i = \hat{S}(\mathsf{sel}_i, \cdot)$ give, splitting w as $(w^{\mathrm{hi}}, w^{\mathrm{lo}})$ with $w^{\mathrm{hi}} \in \{0,1\}^{\nu-\nu_i}$ and $w^{\mathrm{lo}} \in \{0,1\}^{\nu_i}$,

$$\begin{aligned} c_j &= \sum_{y \in \{0,1\}^{\nu_i}} \hat{\mathsf{next}}(z, y)\ \hat{S}(\mathsf{sel}_i, y) \\ &= \sum_{w \in \{0,1\}^{\nu}} \hat{eq}(\mathsf{sel}_i, w^{\mathrm{hi}}) \cdot \hat{\mathsf{next}}(z, w^{\mathrm{lo}})\ \hat{S}(w). \end{aligned}$$

Either way, claim j is equivalent to

$$\sum_{w \in \{0,1\}^\nu} \hat{W}_j(w)\ \hat{S}(w) = c_j,$$

where its *weight* $\hat{W}_j$ is the multilinear polynomial

$$\hat{W}_j(w) = \begin{cases} \hat{eq}\big((\mathsf{sel}_i, z),\, w\big) & \text{(normal claim)} \\ \hat{eq}(\mathsf{sel}_i, w^{\mathrm{hi}}) \cdot \hat{\mathsf{next}}(z, w^{\mathrm{lo}}) & \text{(shift claim)} \end{cases}$$

which the verifier can efficiently evaluate.

Batching into a single PCS opening

The verifier samples a random $\lambda \in \mathbb{F}_q$ and takes the linear combination of the J claims with powers of λ: the single weight $\hat{W}_\lambda = \sum_{j=1}^{J} \lambda^j\, \hat{W}_j$ and target $C_\lambda = \sum_{j=1}^{J} \lambda^j\, c_j$. Except with probability J/q over λ, all J claimed values $c_1, \ldots, c_J$ are correct if and only if

$$\sum_{w \in \{0,1\}^\nu} \hat{W}_\lambda(w)\ \hat{S}(w) = C_\lambda.$$

Since the verifier can efficiently evaluate $\hat{W}_\lambda$, the relation above can be verified as an opening of the inner product PCS (section 3.2.3).

5.1.2 Example

Consider 3 multilinear polynomials $\hat{P}_1, \hat{P}_2, \hat{P}_3$ with 4, 3, and 2 variables respectively. The stacked polynomial $\hat{S}$, which we commit, has $5 = \lceil \log_2(2^4 + 2^3 + 2^2) \rceil$ variables.

- $\hat{P}_1(x_1, x_2, x_3, x_4) = \hat{S}(0, x_1, x_2, x_3, x_4)$
- $\hat{P}_2(x_1, x_2, x_3) = \hat{S}(1, 0, x_1, x_2, x_3)$
- $\hat{P}_3(x_1, x_2) = \hat{S}(1, 1, 0, x_1, x_2)$

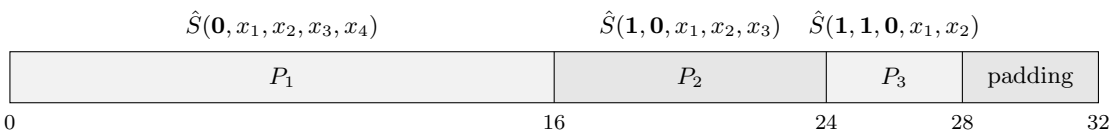


Figure 1: Simple stacking of P_1, P_2, P_3 into a single vector S

5.2 Tables

The execution trace is split across three *tables*: EXECUTION, EXTENSION, and POSEIDON. Each table T is a matrix of base-field elements, with a variable power-of-two height 2^{h_T}, and a fixed number of columns $n^T_{\text{column}} = n^T_{\text{shift}} + n^T_{\text{flat}}$. Each column $\hat{\mathsf{col}}^T_i$ $(i \in [0, n^T_{\text{column}}))$ is interpreted as a multilinear polynomial in h_T variables.

We write h_{EXEC}, h_{POSEIDON}, $h_{\text{EXTENSION}}$ for the log-heights of the three tables, h_{BYTECODE} for the bytecode log-size (so the bytecode has $2^{h_{\text{BYTECODE}}}$ instructions), and h_{MEMORY} for the memory log-size (so the memory has $2^{h_{\text{MEMORY}}}$ cells).

Each VM cycle corresponds to a row of the EXECUTION, each poseidon precompile call corresponds to a row of the POSEIDON, and each extension precompile call with size parameter N (Section 4.7.6) corresponds to N rows of the EXTENSION table.

We restrict their maximal heights, to ensure logup multiplicites never overflow modulo p:

Table	Max rows
EXECUTION	2^{24}
EXTENSION	2^{22}
POSEIDON	2^{22}

5.3 One bus to interact between tables

In order for the tables (Section 5.2) to exchange data, we use a *bus*: a global channel in which tables may PUSH or PULL tuples of $\mathsf{bus\_width} = 16$ field elements in $\mathbb{F}_q$. We enforce the following property: *every tuple is pulled exactly as many times as it is pushed.*

5.3.1 Different types of interaction

Every interaction is a tuple of $\mathsf{bus\_width} = 16$ field elements with a fixed shape:

$$\sigma = \big(\underbrace{d_0, \ldots, d_{N-1}}_{\text{data}}, \underbrace{0, \ldots, 0}_{\text{padding}}, \mathsf{domain\_sep}\big),$$

associated to a *multiplicity*: a value $m \in \mathbb{F}_p$ counting the number of times that tuple is pushed (or pulled). The bus should be balanced: for every σ, the sums of all its *push* multiplicities must equal the sum of all its *pull* multiplicities.

The $N \leq 15$ *data* entries fill the low slots, the last slot always holds the *domain separator* $\mathsf{domain\_sep}$, and the slots in between are zero.

There are 4 different types of interactions:

Memory: Each memory fetch corresponds to a data pair $(\mathsf{address}, \mathsf{value})$, tagged with the domain separator $\mathsf{memory\_sep} = 1$.

Bytecode: Each instruction fetch is the data tuple $(\mathsf{instr}_0, \ldots, \mathsf{instr}_{11}, \mathsf{pc})$, tagged with the domain separator $\mathsf{bytecode\_sep} = 2$.

Poseidon: Each POSEIDON precompile call corresponds to a data tuple (ν_A, ν_B, ν_C) of its three memory pointers, tagged with a non-constant domain separator, that encodes some additional information (compression / permutation etc.), but which is an odd value ≥ 3.

Extension: Each EXTENSION precompile call corresponds to a data tuple (ν_A, ν_B, ν_C) of its three memory pointers, tagged with a non-constant domain separator, that encodes some additional information (dot_product / add / eq etc.), but which is a multiple of 4.

Kind	data entries	$\mathsf{domain\_sep}$
Memory	$(\mathsf{address}, \mathsf{value})$	$\mathsf{memory\_sep} = 1$
Bytecode	$(\mathsf{instr}_0, \ldots, \mathsf{instr}_{11}, \mathsf{pc})$	$\mathsf{bytecode\_sep} = 2$
Poseidon	(ν_A, ν_B, ν_C)	odd, ≥ 3
Extension	(ν_A, ν_B, ν_C)	multiple of 4

A table T may PUSH or PULL various tuples to the common bus.

5.3.2 Special pushes for memory and bytecode

Alongside committing to the execution trace (i.e. the table columns), the prover commits to two access-count multilinear polynomials:

- $\mathsf{memory\_acc} \in \mathbb{F}_p^{2^{h_{\text{MEMORY}}}}$: $\mathsf{memory\_acc}[i]$ = number of pulls of $(i, \mathbf{m}[i], 0_{13}, \mathsf{memory\_sep})$ across all tables.
- $\mathsf{bytecode\_acc} \in \mathbb{F}_p^{2^{h_{\text{BYTECODE}}}}$: $\mathsf{bytecode\_acc}[\mathbf{pc}]$ = number of pulls of $(\mathsf{instr}_0, \dots, \mathsf{instr}_{11}, \mathbf{pc}, 0_2, \mathsf{bytecode\_sep})$ by the execution table.

These columns serve as the multiplicities of the two corresponding bus pushes:

- For each $i \in [0, 2^{h_{\text{MEMORY}}})$: $\textsc{Push}\big(i,\ \mathbf{m}[i],\ 0_{13},\ \mathsf{memory\_sep}\big)$ with multiplicity $\mathsf{memory\_acc}[i]$.
- For each $\mathbf{pc} \in [0, 2^{h_{\text{BYTECODE}}})$: $\textsc{Push}\big(\mathsf{instr}_0(\mathbf{pc}), \dots, \mathsf{instr}_{11}(\mathbf{pc}),\ \mathbf{pc},\ 0_2,\ \mathsf{bytecode\_sep}\big)$ with multiplicity $\mathsf{bytecode\_acc}[\mathbf{pc}]$.

The index slot of these two pushes (i, resp. $\mathbf{pc}$) is *not* committed: it is the position of the entry in the list, so the logup argument below sees it through the MLE of $(0, 1, \dots, 2^h - 1)$ (with $h = h_{\text{MEMORY}}$, resp. h_{BYTECODE}), i.e. the multilinear polynomial $\sum_{j=1}^{h} 2^{h-j} x_j$, which the verifier evaluates on its own. The 12 instruction slots are likewise the MLE of the public bytecode. So the prover only commits to $\mathbf{m}$, $\mathsf{memory\_acc}$ and $\mathsf{bytecode\_acc}$.

Lemma 2 (Indexed lookup)**.** Assume the bytecode is well-formed (Section 4.4), the bus is balanced (i.e. for every tuple, its push and pull multiplicities sum to the same value in $\mathbb{F}_p$), and every tuple is pulled strictly fewer than p times in total (guaranteed by the height bounds of Section 5.2). Then any tuple pulled at least once and of the form $(a, v, 0_{13}, \mathsf{memory\_sep})$ satisfies $a < 2^{h_{\text{MEMORY}}}$ and $v = \mathbf{m}[a]$, and any such tuple of the form $(\mathsf{instr}_0, \dots, \mathsf{instr}_{11}, \mathbf{pc}, 0_2, \mathsf{bytecode\_sep})$ satisfies $\mathbf{pc} < 2^{h_{\text{BYTECODE}}}$ and $(\mathsf{instr}_0, \dots, \mathsf{instr}_{11}) = \mathsf{bytecode}[\mathbf{pc}]$.

Proof. Any tuple pulled at least once is pushed. All pull multiplicities are non-negative integers: the memory and bytecode pulls carry multiplicity 1, and the two precompile pulls carry a multiplicity column built from boolean flags (Sections 5.6 and 5.7). So the total pull multiplicity of such a tuple σ is a positive integer, $< p$ by hypothesis, hence non-zero in $\mathbb{F}_p$. By balance, the total push multiplicity of σ is non-zero as well, so some push carries σ with non-zero multiplicity.

The last slot identifies the pushing family. There are exactly three families of pushes: the memory and bytecode pushes above, whose last slot is the constant 1, resp. 2, and the precompile pushes of the execution table (Section 5.5.4), one per row, of the form $(\nu_A, \nu_B, \nu_C, 0_{12}, \mathsf{aux}_2)$ with multiplicity $\mathsf{flag}_{\mathsf{precompile}}$. So it suffices to prove that $\mathsf{aux}_2 \notin \{1, 2\}$ on every row where $\mathsf{flag}_{\mathsf{precompile}} \neq 0$.

Fix such a row, and write instr for its 12 instruction fields, the last of which, in slot 11, is aux_2. The row pulls its own instruction, with multiplicity 1, as the tuple $\tau = (\mathsf{instr}, \mathbf{pc}, 0_2, \mathsf{bytecode\_sep})$. By the previous step some push carries τ, and the last slot of τ is $2 \neq 1$, so that push is a bytecode or a precompile one:

- a bytecode push: then $\mathsf{instr} = \mathsf{bytecode}[\mathbf{pc}]$, so aux_2 is the last field of a well-formed instruction, namely 0 for `ADD`, `MUL`, `DEREF` and `JUMP`, an odd value ≥ 3 for `POSEIDON`, a multiple of 4 for `EXTENSION` (Section 5.5.2);
- a precompile push: its slots $3, \dots, 14$ are 0, and comparing slot 11 gives $\mathsf{aux}_2 = 0$.

In both cases $\mathsf{aux}_2 \notin \{1, 2\}$. A push carrying a tuple whose last slot is 1, resp. 2, is therefore a memory, resp. bytecode, push.

Conclusion. Let $\sigma = (a, v, 0_{13}, \mathsf{memory\_sep})$ be pulled at least once. Its last slot being 1, the push carrying it is a memory push, i.e. $\sigma = (i, \mathbf{m}[i], 0_{13}, \mathsf{memory\_sep})$ for some $i \in [0, 2^{h_{\text{MEMORY}}})$; comparing the first two slots, $a = i < 2^{h_{\text{MEMORY}}}$ and $v = \mathbf{m}[a]$. The bytecode case is identical, with $\mathsf{bytecode\_sep}$ in place of $\mathsf{memory\_sep}$ and the 12 instruction slots in place of the single value slot. □

5.3.3 Balanced bus proved via logup

Hashing tuples to a single field element. For some $\vec{\beta} = (\beta_1, \ldots, \beta_\ell) \in \mathbb{F}_q^\ell$ sampled uniformly at random, we define:

$$\pi_{\vec{\beta}}(\sigma) \;=\; \sum_{i=0}^{\mathsf{bus\_width}-1} \sigma_i \cdot \hat{eq}\big(\vec{\beta},\, [i]^\ell_{\text{bits}}\big) \;=\; \hat{\sigma}(\vec{\beta}).$$

By Lemma 1 applied to the (non-zero) multilinear polynomial $\hat{\sigma} - \hat{\sigma'}$ in ℓ variables, two distinct tuples collide with probability at most ℓ/q.

Bus balance as a rational identity. Enumerate the bus interactions across all tables as a list of pairs $(\sigma_k, m_k)_{1 \le k \le K}$, where $\sigma_k \in \mathbb{F}_p^{\mathsf{bus\_width}}$ is the pushed (or pulled) tuple data and $m_k \in \mathbb{F}_p$ is its signed multiplicity (positive for PUSH, negative for PULL) (where we naturally embed the integers $(-p, p)$ into $\mathbb{F}_p$). Assume that, for every tuple σ, both the total pushes and the total pulls of σ are strictly less than p. Then the bus is balanced if and only if, for every σ, $\sum_{k\,:\,\sigma_k = \sigma} m_k = 0$, which, for $\vec{\beta} \xleftarrow{\$} \mathbb{F}_q^\ell$ and $\gamma \xleftarrow{\$} \mathbb{F}_q$, is equivalent up to completeness error K/q and soundness error $K\ell/q$ to:

$$\sum_{k=1}^{K} \frac{m_k}{\gamma - \pi_{\vec{\beta}}(\sigma_k)} \;=\; 0. \tag{1}$$

Proof. (Inspired by Soundness of Interactions via LogUp) Define the $\mathbb{F}_p$-valued multiset $M(\sigma) = \sum_{k\,:\,\sigma_k = \sigma} m_k$, with support $S = \{\sigma : M(\sigma) \neq 0\}$, of size $s = |S|$. The bus is balanced iff $S = \emptyset$.

Completeness. If $S = \emptyset$, then $M(\sigma) = 0$ for every σ, so the rational sum is identically zero wherever it is defined. The sum is undefined when γ coincides with some pole $\pi_{\vec{\beta}}(\sigma_k)$, which over uniform γ happens with probability at most K/q: the protocol's *completeness error*.

Soundness. Suppose $S \neq \emptyset$, and define

$$N(\vec{\beta}, \gamma) \;:=\; \sum_{\sigma \in S} M(\sigma) \prod_{\sigma' \in S \setminus \{\sigma\}} \big(\gamma - \pi_{\vec{\beta}}(\sigma')\big), \qquad Q(\vec{\beta}, \gamma) \;:=\; \prod_{\sigma \in S} \big(\gamma - \pi_{\vec{\beta}}(\sigma)\big).$$

Over a common denominator, the rational sum equals N/Q. Verifier acceptance requires this fraction to be defined and equal to zero, hence forces $N(\vec{\beta}, \gamma) = 0$.

Total degree of N. The degree of N, interpreted as multivariable polynomial in γ and $\vec{\beta}$, is at most $(s-1)\ell \le K\ell$.

Non-vanishing of N. Pick any $\sigma^\star \in S$ (so $M(\sigma^\star) \neq 0$). To show N is not the zero polynomial in $\mathbb{F}_q[\vec{\beta}, \gamma]$, substitute the variable γ by the polynomial $\pi_{\vec{\beta}}(\sigma^\star) \in \mathbb{F}_q[\vec{\beta}]$. If N were the zero polynomial its image would also be zero. For every $\sigma \in S$ with $\sigma \neq \sigma^\star$, the product $\prod_{\sigma' \in S \setminus \{\sigma\}}$ contains the factor $\gamma - \pi_{\vec{\beta}}(\sigma^\star)$ (since $\sigma^\star \in S \setminus \{\sigma\}$), which the substitution sends to 0; only the $\sigma = \sigma^\star$ summand survives, giving

$$N\big(\vec{\beta},\, \pi_{\vec{\beta}}(\sigma^\star)\big) \;=\; M(\sigma^\star) \prod_{\sigma' \in S \setminus \{\sigma^\star\}} (\hat{\sigma}^\star - \hat{\sigma}')(\vec{\beta}) \;\in\; \mathbb{F}_q[\vec{\beta}].$$

Each factor in this product is the MLE of the non-zero vector $\sigma^\star - \sigma' \in \mathbb{F}_p^{\mathsf{bus\_width}}$, hence a non-zero polynomial in $\vec{\beta}$. Given that $M(\sigma^\star) \neq 0$, N is non-zero in $\mathbb{F}_q[\vec{\beta}, \gamma]$.

By Lemma 1,

$$\Pr_{\vec{\beta}, \gamma}[N(\vec{\beta}, \gamma) = 0] \;\le\; K\,\ell/q. \qquad \square$$

5.3.4 Logup-GKR

The prover can convince the verifier of the validity of equation (1) efficiently via GKR [11], as long as the verifier can efficiently evaluate the MLEs of $(m_k)_{k \in [1,K]}$ and $(\pi_{\vec{\beta}}(\sigma_k))_{k \in [1,K]}$ (assuming K is a power-of-two, which we can always guarantee up to some padding: zeros for numerators, ones for denominators).

We briefly recall the GKR construction in this context.

Assume the prover wants to convince the verifier $\sum_{b \in \{0,1\}^n} \frac{\hat{num}(b)}{\hat{den}(b)} = \kappa$ for some $\hat{num}, \hat{den} \in \mathbb{F}_q^{<2}[X_1, \ldots, X_n]$ efficiently evaluable and $\kappa \in \mathbb{F}_q$.

Define the following protocol $GKR(n)$:

- If $n = 0$, the verifier can evaluate the sum by evaluting $\hat{num}$ and $\hat{den}$ once each, at a common point (the empty point, since both are constant polynomial).
- If $n > 0$:
  - Write $\kappa = \sum_{b\in\{0,1\}^{n-1}} \frac{\hat{num}'(b)}{\hat{den}'(b)}$, where num' is the MLE of $(\hat{num}(0,b)\cdot\hat{den}(1,b)+\hat{num}(1,b)\cdot\hat{den}(0,b))_{b\in\{0,1\}^{n-1}}$ and $\hat{den}'$ is the MLE of $(\hat{den}(0,b)\cdot\hat{den}(1,b))_{b\in\{0,1\}^{n-1}}$
  - recursively call $GKR(n-1)$ on this new sum, which will return 2 claims of the form: $\hat{num}'(\alpha) = \beta^{num}$ and $\hat{den}'(\alpha) = \beta^{den}$ with $\alpha \in \mathbb{F}_q^{n-1}, \beta^{num}, \beta^{den} \in \mathbb{F}_q$.
  - Sample $\gamma \in \mathbb{F}_q$ and run the sumcheck protocol on:

$$\sum_{b\in\{0,1\}^{n-1}} eq(\alpha,b)\big[(\hat{num}(0,b)\cdot\hat{den}(1,b)+\hat{num}(1,b)\cdot\hat{den}(0,b))+\gamma\cdot\hat{den}(0,b)\cdot\hat{den}(1,b)\big] = \beta^{num}+\gamma\cdot\beta^{den}$$

  - Let $\eta \in \mathbb{F}_q^{n-1}$ be the sumcheck challenges. The prover sends $num_0, num_1, den_0, den_1$, respectively equal to $\hat{num}(0,b), \hat{num}(1,b), \hat{den}(0,b), \hat{den}(1,b)$ in the honest case. This enables the verifier to evaluate the expression within the sum above, and to conclude the sumcheck.
  - Sample $\delta \in \mathbb{F}_q$, and conclude the GKR step by outputing the following claims (that remains to be proven): $\hat{num}(\delta,\eta) \stackrel{?}{=} (1-\delta)\cdot num_0 + \delta\cdot num_1$ and $\hat{den}(\delta,\eta) \stackrel{?}{=} (1-\delta)\cdot den_0 + \delta\cdot den_1$.

5.3.5 Stacking within logup

In practice, the fingerprinted data $\pi_{\vec{\beta}}(\sigma_k)$ and multiplicities m_k come from multiple columns, of potentially different sizes. We stack them in a similar way than Section 5.1. TODO give more details

5.4 AIR constraints

Every table T has a list of $n_T^{\text{constraints}}$ AIR (Algebraic Intermediate Representation) constraints. Each AIR constraint C_k^T ($k \in [0, n^T_{\text{constraints}})$) is a multivariate polynomial in $n^T_{\text{column}} + n^T_{\text{shift}}$ variables: one variable per column (taking its row-r value $\mathsf{col}_i^T(r)$ as input), plus one variable per shifted column (taking $\mathsf{shift}(\mathsf{col}_j^T)[r]$ as input). The constraint is respected by the table's trace if, for every row $r \in [0, 2^{h_T})$,

$$C_k^T\big(\mathsf{col}_0^T(r), \ldots, \mathsf{col}^T_{n^T_{\text{column}}-1}(r),\ \mathsf{shift}(\mathsf{col}_0^T)[r], \ldots, \mathsf{shift}(\mathsf{col}^T_{n^T_{\text{shift}}-1})[r]\big) = 0.$$

5.4.1 ZeroCheck: Proving AIR constraints via sumcheck

For each table T, we want to convince the verifier that every constraint C_k^T vanishes on every row of the trace. We proceed in three steps.

Batching. The verifier samples a single $\mu \in \mathbb{F}_q$, and we form, for each table T, the batched constraint

$$H_T \ := \sum_{k=0}^{n^T_{\text{constraints}}-1} \mu^{\iota_T+k}\, C_k^T,$$

where $\iota_T := \sum_{T'<T} n^{T'}_{\text{constraints}}$ is the offset of T in a global ordering of all constraints across all tables (so consecutive powers $1, \mu, \mu^2, \ldots$ are distributed across all constraints).

Completeness: if every C_k^T vanishes on every row, so does each H_T.

Soundness: if some C_k^T fails to vanish at some row $\vec{b}^\star$, then $\mu \mapsto H_T(\vec{b}^\star)$ is a non-zero univariate polynomial of degree $< N := \sum_T n^T_{\text{constraints}}$, so $H_T(\vec{b}^\star) \neq 0$ except with soundness error N/q over uniform μ, via Schwartz-Zippel (lemma 1).

Zerocheck. Let $f_T \in \mathbb{F}_q^{2^{h_T}}$ be the vector whose $\vec{b}$-th coordinate is

$$f_T(\vec{b}) \ := \ H_T\big(\hat{\mathsf{col}}_0^T(\vec{b}), \ldots, \hat{\mathsf{col}}^T_{n^T_{\text{column}}-1}(\vec{b}), \hat{\mathsf{shift}}(\mathsf{col}_0^T)(\vec{b}), \ldots, \hat{\mathsf{shift}}(\mathsf{col}^T_{n^T_{\text{shift}}-1})(\vec{b})\big),$$

so the claim is $f_T \equiv 0$. The verifier samples $\vec{r} \in \mathbb{F}_q^{h_T}$. By Lemma 1 applied to the multilinear polynomial (in h_T variables) $\hat{f}_T$, with soundness error h_T/q it suffices to prove that $\hat{f}_T(\vec{r}) = 0$, i.e.,

$$\sum_{\vec{b} \in \{0,1\}^{h_T}} \hat{eq}(\vec{b}, \vec{r}) \cdot f_T(\vec{b}) \;=\; 0.$$

Sumcheck. Run the standard sumcheck protocol [4] on the sum above. After h_T rounds with challenges $\vec{\beta} \in \mathbb{F}_q^{h_T}$, the verifier must check the summand at $\vec{b} = \vec{\beta}$: $\hat{eq}(\vec{\beta}, \vec{r})$ is computed directly, and the column and shifted-column evaluations are claimed by the prover and verified as described in Section 5.1.

Beyond consecutive sampling of the challenge $\vec{r}$. Prover and verifier may engage in arbitrary auxiliary sub-protocols between successive r_i samplings; what matters for soundness is only that each r_i is sampled uniformly at random and after the trace commitment.

5.4.2 Batching many ZeroChecks together

Instead of running the zerocheck above individually for each table, we merge them into a single *back-loaded* sumcheck of $h_{\max} := \max_T h_T$ rounds, using a shared random challenge $\vec{r} \in \mathbb{F}_q^{h_{\max}}$. For each table T, let $\vec{r}_T \in \mathbb{F}_q^{h_T}$ be the suffix of $\vec{r}$ of length h_T, write $\vec{X}_{\geq k} := (X_k, \ldots, X_{h_{\max}-1})$, and define $G_T \in \mathbb{F}_q[X_0, \ldots, X_{h_{\max}-1}]$ by

$$G_T(\vec{X}) \;:=\; \Big(\prod_{i<h_{\max}-h_T} X_i\Big) \cdot \hat{eq}\big(\vec{X}_{\geq h_{\max}-h_T},\, \vec{r}_T\big) \cdot f_T\big(\vec{X}_{\geq h_{\max}-h_T}\big).$$

We run a single sumcheck claiming

$$\sum_{\vec{b} \in \{0,1\}^{h_{\max}}} \sum_T G_T(\vec{b}) \;=\; 0,$$

which is equivalent to the conjunction of the three per-table zerocheck claims.

5.4.3 Attaching evaluation claims

Suppose, in addition to the AIR constraints, we also want to verify evaluation claims of the form $\hat{V}(\vec{r}_T) = v$, where V is a polynomial expression in the columns of T (sometimes called *virtual column*, i.e. that is not committed). One way to see it: treat each such V as just another constraint of T, batched in alongside the C_k^T's with its own power of μ in the global ordering: the only difference is that its expected sum is no longer 0. Listing all eval claims across the three tables as $(v_1, \ldots, v_M)$ with respective global indices $(\iota_1, \ldots, \iota_M)$, the zerocheck target shifts from 0 to

$$\sum_{i=1}^{M} \mu^{\iota_i}\, v_i.$$

5.5 EXECUTION table

5.5.1 Columns

Each row corresponds to a VM cycle. It has $n_{cols}^{execution} = 20$ columns.

- **pc** (program counter)
- **fp** (frame pointer)
- addr_A, addr_B, addr_C (memory indexes)
- value_A, value_B, value_C (corresponding memory values)
- 12 columns encoding the instruction being executed:
  - flag_A, flag_B, flag_C, $\mathsf{flag}_{\mathbf{fp}}^C$, $\mathsf{flag}_{\mathsf{mul}}$, $\mathsf{flag}_{\mathsf{jump}}$, $\mathsf{flag}_{\mathbf{fp}}^{AB} \in \{0, 1\}$

– $\mathsf{aux}_1 \in \{0,1,2\}$ (aux_1 handles both ADD and DEREF)

– $\mathsf{operand}_A$, $\mathsf{operand}_B$, $\mathsf{operand}_C, \mathsf{aux}_2 \in \mathbb{F}_p$

Note. $\mathsf{flag}_{\mathbf{fp}}^{AB}$ is only used by precompile instructions; for ADD, MUL, DEREF, and JUMP it is always 0. We can reuse it to enrich the ISA. But is it worth it?

5.5.2 Opcode encoding

We first derive three values ν_A, ν_B, ν_C from the instruction columns:

- $\nu_A = \mathsf{flag}_A \cdot \mathsf{operand}_A + (1 - \mathsf{flag}_A - \mathsf{flag}_{\mathbf{fp}}^{AB}) \cdot \mathsf{value}_A + \mathsf{flag}_{\mathbf{fp}}^{AB} \cdot (\mathbf{fp} + \mathsf{operand}_A)$
- $\nu_B = \mathsf{flag}_B \cdot \mathsf{operand}_B + (1 - \mathsf{flag}_B - \mathsf{flag}_{\mathbf{fp}}^{AB}) \cdot \mathsf{value}_B + \mathsf{flag}_{\mathbf{fp}}^{AB} \cdot (\mathbf{fp} + \mathsf{operand}_B)$
- $\nu_C = \mathsf{flag}_C \cdot \mathsf{operand}_C + (1 - \mathsf{flag}_C - \mathsf{flag}_{\mathbf{fp}}^{C}) \cdot \mathsf{value}_C + \mathsf{flag}_{\mathbf{fp}}^{C} \cdot (\mathbf{fp} + \mathsf{operand}_C)$

For an ISA operand pair $(x^{\mathsf{addr}}, x) \in \{\texttt{imm}, \texttt{mem}, \texttt{fp\_rel}\} \times \mathbb{F}_p$ to be assigned to ν_X ($X \in \{A, B, C\}$), we write $\nu_X \leftarrow (x^{\mathsf{addr}}, x)$ for the column assignment defined by

x^{addr}	columns set	resulting ν_X
`imm`	$\mathsf{flag}_X = 1,\ \mathsf{operand}_X = x$	x
`mem`	$\mathsf{flag}_X = 0,\ \mathsf{operand}_X = x$	$\mathbf{m}[\mathbf{fp} + x]$
`fp_rel`	$\mathsf{flag}_{\mathbf{fp}}^{C} = 1,\ \mathsf{operand}_C = x$ ($X = C$ only)	$\mathbf{fp} + x$

For precompiles, when $\alpha^{\mathsf{addr}} = \beta^{\mathsf{addr}} = \texttt{fp\_rel}$, the encoding uses the simultaneous mode instead: $\mathsf{flag}_{\mathbf{fp}}^{AB} = 1$, $\mathsf{operand}_A = \alpha$, $\mathsf{operand}_B = \beta$, yielding $\nu_A = \mathbf{fp} + \alpha$ and $\nu_B = \mathbf{fp} + \beta$. All columns not explicitly set below are zero.

- $\texttt{ADD}[\alpha^{\mathsf{addr}}, \alpha, \beta^{\mathsf{addr}}, \beta, \gamma^{\mathsf{addr}}, \gamma]$ (semantics $\nu_A + \nu_C = \nu_B$): $\mathsf{aux}_1 = 1$; $\nu_A \leftarrow (\alpha^{\mathsf{addr}}, \alpha)$, $\nu_B \leftarrow (\beta^{\mathsf{addr}}, \beta)$, $\nu_C \leftarrow (\gamma^{\mathsf{addr}}, \gamma)$.
- $\texttt{MUL}[\alpha^{\mathsf{addr}}, \alpha, \beta^{\mathsf{addr}}, \beta, \gamma^{\mathsf{addr}}, \gamma]$ (semantics $\nu_A \cdot \nu_C = \nu_B$): $\mathsf{flag}_{\mathsf{mul}} = 1$; $\nu_A \leftarrow (\alpha^{\mathsf{addr}}, \alpha)$, $\nu_B \leftarrow (\beta^{\mathsf{addr}}, \beta)$, $\nu_C \leftarrow (\gamma^{\mathsf{addr}}, \gamma)$.
- $\texttt{DEREF}[\alpha, \beta, \gamma^{\mathsf{addr}}, \gamma]$ (semantics $\mathbf{m}[\mathbf{m}[\mathbf{fp} + \alpha] + \beta] = \nu_C$): $\mathsf{aux}_1 = 2$; $\mathsf{flag}_A = 0$, $\mathsf{operand}_A = \alpha$ (so $\nu_A = \mathbf{m}[\mathbf{fp} + \alpha]$); $\mathsf{flag}_B = 1$, $\mathsf{operand}_B = \beta$; $\nu_C \leftarrow (\gamma^{\mathsf{addr}}, \gamma)$.
- $\texttt{JUMP}[\alpha^{\mathsf{addr}}, \alpha, \beta^{\mathsf{addr}}, \beta, \gamma^{\mathsf{addr}}, \gamma]$ (condition / dest / updated-fp): $\mathsf{flag}_{\mathsf{jump}} = 1$; $\nu_A \leftarrow (\alpha^{\mathsf{addr}}, \alpha)$, $\nu_B \leftarrow (\beta^{\mathsf{addr}}, \beta)$, $\nu_C \leftarrow (\gamma^{\mathsf{addr}}, \gamma)$.
- $\texttt{POSEIDON\_OP}[\alpha^{\mathsf{addr}}, \alpha, \beta^{\mathsf{addr}}, \beta, \gamma^{\mathsf{addr}}, \gamma, \texttt{permute}, \texttt{out8}, \texttt{out4}, \delta]$: $\nu_A \leftarrow (\alpha^{\mathsf{addr}}, \alpha)$, $\nu_B \leftarrow (\beta^{\mathsf{addr}}, \beta)$, $\nu_C \leftarrow (\gamma^{\mathsf{addr}}, \gamma)$, and

  $$\mathsf{aux}_2 = 3 + 2\,\texttt{permute} + 4\,\texttt{out8} + 8\,[\delta \neq \bot] + 16\,\delta\,[\delta \neq \bot].$$

  (`out4` is omitted from aux_2; see the POSEIDON table below.)
- $\texttt{EXTENSION\_OP}[\alpha^{\mathsf{addr}}, \alpha, \beta^{\mathsf{addr}}, \beta, \gamma^{\mathsf{addr}}, \gamma, \mathsf{op}, \mathsf{flag\_be}, N]$: $\nu_A \leftarrow (\alpha^{\mathsf{addr}}, \alpha)$, $\nu_B \leftarrow (\beta^{\mathsf{addr}}, \beta)$, $\nu_C \leftarrow (\gamma^{\mathsf{addr}}, \gamma)$, and

  $$\mathsf{aux}_2 = 4\,\mathsf{flag\_be} + 8\,\mathsf{flag\_add} + 16\,\mathsf{flag\_dot\_product} + 32\,\mathsf{flag\_eq} + 64N,$$

  where $(\mathsf{flag\_add}, \mathsf{flag\_dot\_product}, \mathsf{flag\_eq})$ is the one-hot encoding of op.

5.5.3 AIR constraints

Constraints have degree 5. The values ν_A, ν_B, ν_C are as defined in Section 5.5.2.

Let P_0, P_1, P_2 be the Lagrange basis polynomials at $0, 1, 2$:

$$P_0(x) = (x-1)(x-2)/2, \qquad P_1(x) = x(2-x), \qquad P_2(x) = x(x-1)/2.$$

We define the virtual (non-committed) selectors

- $\mathsf{flag}_{\mathsf{add}} = P_1(\mathsf{aux}_1)$ (equals 1 when $\mathsf{aux}_1 = 1$, else 0 on $\{0, 2\}$),
- $\mathsf{flag}_{\mathsf{deref}} = P_2(\mathsf{aux}_1)$ (equals 1 when $\mathsf{aux}_1 = 2$, else 0 on $\{0, 1\}$),
- $\mathsf{flag}_{\mathsf{precompile}} = 1 - (\mathsf{flag}_{\mathsf{add}} + \mathsf{flag}_{\mathsf{mul}} + \mathsf{flag}_{\mathsf{deref}} + \mathsf{flag}_{\mathsf{jump}})$.

$\mathsf{flag}_{\mathsf{precompile}}$ is the selector for the precompile bus push (see Section 5.5.4).

Addressing constraints.

- $(1 - \mathsf{flag}_A - \mathsf{flag}_{\mathbf{fp}}^{AB}) \cdot (\mathsf{address}_A - (\mathbf{fp} + \mathsf{operand}_A)) = 0$
- $(1 - \mathsf{flag}_B - \mathsf{flag}_{\mathbf{fp}}^{AB}) \cdot (\mathsf{address}_B - (\mathbf{fp} + \mathsf{operand}_B)) = 0$
- $(1 - \mathsf{flag}_C - \mathsf{flag}_{\mathbf{fp}}^{C}) \cdot (\mathsf{address}_C - (\mathbf{fp} + \mathsf{operand}_C)) = 0$

Per-opcode semantic constraints.

- `ADD`: $\mathsf{flag}_{\mathsf{add}} \cdot (\nu_B - (\nu_A + \nu_C)) = 0$.
- `MUL`: $\mathsf{flag}_{\mathsf{mul}} \cdot (\nu_B - \nu_A \cdot \nu_C) = 0$.
- `DEREF`: $\mathsf{flag}_{\mathsf{deref}} \cdot (\mathsf{addr}_B - (\mathsf{value}_A + \mathsf{operand}_B)) = 0$ and $\mathsf{flag}_{\mathsf{deref}} \cdot (\mathsf{value}_B - \nu_C) = 0$.
- `JUMP`: let $J = \mathsf{flag}_{\mathsf{jump}} \cdot \nu_A$ (so $J = 1$ on a taken jump, $J = 0$ otherwise). Then:
  - $J \cdot (1 - \nu_A) = 0$ (forces $\nu_A \in \{0, 1\}$ on jump rows)
  - $J \cdot (\mathrm{next}(\mathbf{pc}) - \nu_B) = 0$ and $(1 - J) \cdot (\mathrm{next}(\mathbf{pc}) - (\mathbf{pc} + 1)) = 0$
  - $J \cdot (\mathrm{next}(\mathbf{fp}) - \nu_C) = 0$ and $(1 - J) \cdot (\mathrm{next}(\mathbf{fp}) - \mathbf{fp}) = 0$

Well-formed-bytecode assumption. The constraints above assume each instruction is well-formed.
TODO: Verify and (formally) prove completeness + soundness.

5.5.4 Bus interactions

Memory pulls (three per row). For each $X \in \{A, B, C\}$:

$$\textsc{Pull}\big(\mathsf{addr}_X,\ \mathsf{value}_X,\ 0_{13},\ \mathsf{memory\_sep}\big) \quad \text{with multiplicity 1.}$$

Bytecode pull (one per row).

$$\textsc{Pull}\big(\mathsf{instr},\ \mathbf{pc},\ 0_2,\ \mathsf{bytecode\_sep}\big) \quad \text{with multiplicity 1.}$$

where we write:

$$\mathsf{instr} \ = \ \big(\mathsf{operand}_A, \mathsf{operand}_B, \mathsf{operand}_C, \mathsf{flag}_A, \mathsf{flag}_B, \mathsf{flag}_C, \mathsf{flag}_{\mathbf{fp}}^{C}, \mathsf{flag}_{\mathbf{fp}}^{AB}, \mathsf{flag}_{\mathsf{mul}}, \mathsf{flag}_{\mathsf{jump}}, \mathsf{aux}_1, \mathsf{aux}_2\big)$$

for the 12 instruction columns.

Precompile push (conditional).

$$\textsc{Push}\big(\nu_A,\ \nu_B,\ \nu_C,\ 0_{12},\ \mathsf{aux}_2\big) \quad \text{with multiplicity } \mathsf{flag}_{\mathsf{precompile}} \text{ (see Section 5.5.3),}$$

where aux_2 encodes a call to either the POSEIDON or EXTENSION table.

5.6 POSEIDON table

The Poseidon table runs one `POSEIDON_OP` precompile call (Section 4.7.5) per row, pulling the call $(\nu_A, \nu_B, \nu_C, 0_{12}, \mathsf{aux}_2)$ from the precompile bus. The call's compile-time parameters are recorded in the row as the boolean flags $\mathsf{flag\_permute}, \mathsf{flag\_out8}, \mathsf{flag\_out4}, \mathsf{flag\_left}$ and the offset $\mathsf{offset\_left} \in \mathbb{F}_p$, with $\mathsf{flag\_left} = [\delta \neq \bot]$ and $\mathsf{offset\_left} = \delta$ (so $\mathsf{flag\_left} = 1$ replaces the first four left-input elements by a hardcoded prefix). The pair $(\mathsf{flag\_out8}, \mathsf{flag\_out4})$ encodes the output length — $(1, 0) \mapsto 8$, $(0, 1) \mapsto 4$, $(0, 0) \mapsto 16$ — so the four valid $(\mathsf{permute}, \text{output})$ modes are: compression 8 ($\mathsf{permute}{=}0, \mathsf{out8}{=}1$), compression 4 ($\mathsf{permute}{=}0, \mathsf{out4}{=}1$), permutation 16 ($\mathsf{permute}{=}1, \mathsf{out16}$), and permutation 8 ($\mathsf{permute}{=}1, \mathsf{out8}{=}1$).

5.6.1 Columns

It has $n_{cols}^{poseidon} = 110$ columns.

- **Control / pointers (10):** the multiplicity (for precompile bus-interaction) $\mathsf{multiplicity}$ (1 on active rows, 0 on padding); the boolean mode flags $\mathsf{flag\_permute}$, $\mathsf{flag\_out8}$, $\mathsf{flag\_out4}$, $\mathsf{flag\_left}$ and the offset $\mathsf{offset\_left}$; the right-input and output pointers ν_B, ν_C; and two derived left addresses $\mathsf{addr}_{\text{left}}^{\text{lo}}$, $\mathsf{addr}_{\text{left}}^{\text{hi}}$, equal to $(\nu_A, \nu_A{+}4)$ if $\mathsf{flag\_left}{=}0$ and to $(\mathsf{offset\_left}, \nu_A)$ if $\mathsf{flag\_left}{=}1$.
- **Input (16):** $\mathsf{input}_0, \ldots, \mathsf{input}_{15}$, the permutation input $\texttt{left} \,\|\, \texttt{right}$ (so $\mathsf{input}_{0..8} = \texttt{left}$ and $\mathsf{input}_{8..16} = \texttt{right}$).
- **Intermediate rounds (68):** the full state is committed only after each *pair* of full rounds: $\mathsf{full}_1, \mathsf{full}_2$ (16 columns each) after the two pairs making up the 4 initial full rounds, and full_3 (16 columns) after the first pair of the 4 final full rounds (the last pair feeds the output columns directly). Plus $\mathsf{sbox}_1, \ldots, \mathsf{sbox}_{20}$, the single S-box output (the cube of the first state coordinate) committed at each partial round.
- **Output (16):** two halves $\mathsf{out}_{\text{lo}}, \mathsf{out}_{\text{hi}}$ (8 columns each). out_{lo} holds the written first half — the feed-forward sum $s[0..8] + \texttt{left}$ in compression, or $s[0..8]$ in permutation; out_{hi} holds $s[8..16]$ and is constrained only by the full 16-element permutation. Cells the AIR leaves unconstrained ($\mathsf{out}_{\text{lo}}[4..8]$ when the output is 4; out_{hi} when it is not 16) are set to the corresponding memory values so the 16-cell output lookup balances.

Non-committed (virtual): ν_A (left-input pointer) and aux_2 (the precompile-data domain separator, see below).

5.6.2 Bus interactions

Memory pulls. Each committed input/output cell is tied to memory by a $\textsc{Pull}\big(a,\, c,\, 0_{13},\, \mathsf{memory\_sep}\big)$ of multiplicity 1 (address a holding value c):

- input_i at $\mathsf{addr}_{\text{left}}^{\text{lo}} + i$ and input_{4+i} at $\mathsf{addr}_{\text{left}}^{\text{hi}} + i$, for $i \in [0, 4)$ (the left half);
- input_{8+i} at $\nu_B + i$, for $i \in [0, 8)$ (the right half);
- $\mathsf{out}_{\text{lo}}[i]$ at $\nu_C + i$ and $\mathsf{out}_{\text{hi}}[i]$ at $\nu_C + 8 + i$, for $i \in [0, 8)$.

Precompile pull. The table pulls the call tuple

$$\textsc{Pull}\big(\nu_A,\, \nu_B,\, \nu_C,\, 0_{12},\, \mathsf{aux}_2\big) \quad \text{with multiplicity } \mathsf{multiplicity},$$

matching the push issued by the execution table (Section 5.5.4), with domain separator

$$\mathsf{aux}_2 = 3 + 2\,\mathsf{flag\_permute} + 4\,\mathsf{flag\_out8} + 8\,\mathsf{flag\_left} + 16\,\mathsf{flag\_left} \cdot \mathsf{offset\_left}$$

Although $\mathsf{flag\_out4}$ is absent, aux_2 is injective over valid modes. It does not wrap mod p: $\mathsf{flag\_left} = 1$ forces $\mathsf{offset\_left} < 2^{h_{\text{MEMORY}}} \le 2^{26}$ (otherwsie the memory lookup is invalid), so $16\,\mathsf{offset\_left} \le 2^{30} < p$. Hence $\mathsf{aux}_2 \bmod 8 = 3 + 2\,\mathsf{flag\_permute} + 4\,\mathsf{flag\_out8}$ recovers $\mathsf{flag\_permute}, \mathsf{flag\_out8}$ (the values $\{0, 2, 4, 6\}$ are distinct); the term $8\,\mathsf{flag\_left} + 16\,\mathsf{flag\_left} \cdot \mathsf{offset\_left}$ recovers $\mathsf{flag\_left}$ and $\mathsf{offset\_left}$; and the constraints below pin $\mathsf{flag\_out4} = (1 - \mathsf{flag\_permute})(1 - \mathsf{flag\_out8})$. So every flag is determined by aux_2.

5.6.3 AIR constraints

Constraints have degree 10.

Write $\nu_A = \mathsf{addr}_{\text{left}}^{\text{hi}} - (1 - \mathsf{flag\_left}) \cdot 4$ for the (virtual) left-input pointer, and let $\sigma \in \mathbb{F}_p^{16}$ denote the running state of the recomputed permutation.

Booleanity (5). $\mathsf{multiplicity}$, $\mathsf{flag\_permute}$, $\mathsf{flag\_out8}$, $\mathsf{flag\_out4}$, $\mathsf{flag\_left}$ are each boolean: $x \cdot (1 - x) = 0$.

Mode separation (3).

- $\mathsf{flag\_permute} \cdot \mathsf{flag\_out4} = 0$ (permutation never produces a 4-element output);
- $\mathsf{flag\_out8} \cdot \mathsf{flag\_out4} = 0$ (output length is one of $\{4, 8, 16\}$);
- $(1 - \mathsf{flag\_permute})(1 - \mathsf{flag\_out8})(1 - \mathsf{flag\_out4}) = 0$ (compression never produces a 16-element output)

Left-address binding (2).

- $\mathsf{flag\_left} \cdot (\mathsf{offset\_left} - \mathsf{addr}^{\mathrm{lo}}_{\mathrm{left}}) = 0$ (when $\mathsf{flag\_left} = 1$: $\mathsf{addr}^{\mathrm{lo}}_{\mathrm{left}} = \mathsf{offset\_left}$, $\mathsf{addr}^{\mathrm{hi}}_{\mathrm{left}} = \nu_A$);
- $(1 - \mathsf{flag\_left}) \cdot (\nu_A - \mathsf{addr}^{\mathrm{lo}}_{\mathrm{left}}) = 0$ (when $\mathsf{flag\_left} = 0$: $\mathsf{addr}^{\mathrm{lo}}_{\mathrm{left}} = \nu_A$, $\mathsf{addr}^{\mathrm{hi}}_{\mathrm{left}} = \nu_A + 4$).

Permutation (68). Starting from $\sigma = \mathsf{input}$, recompute the Poseidon permutation (round constants, S-box $x \mapsto x^3$, and the MDS / sparse partial-round layers; we omit the Poseidon-internal details), and check it against the committed round witnesses:

- *full-round pairs (48):* after each committed pair, the 16 recomputed coordinates equal the witness — $\sigma = \mathsf{full}_1$, then full_2 (the two initial pairs), then full_3 (the first final pair);
- *partial rounds (20):* at partial round k, $\mathsf{sbox}_k = (\sigma_0)^3$ (and σ_0 is then set to sbox_k, keeping the constraint low-degree).

Output (16). Let σ be the state after the *last* (uncommitted) final full-round pair, and write the per-cell value $v_i = \sigma_i + (1 - \mathsf{flag\_permute}) \cdot \mathsf{input}_i$ (the feed-forward sum in compression, the raw state in permutation). For each $i \in [0, 8)$:

- $v_i - \mathsf{out}_{\mathrm{lo}}[i] = 0$ for $i \in [0, 4)$ (always constrained), and $(1 - \mathsf{flag\_out4}) \cdot \big(v_i - \mathsf{out}_{\mathrm{lo}}[i]\big) = 0$ for $i \in [4, 8)$ (the low half is real unless the output is 4);
- $(1 - \mathsf{flag\_out8} - \mathsf{flag\_out4}) \cdot \big(\sigma_{i+8} - \mathsf{out}_{\mathrm{hi}}[i]\big) = 0$ ($\mathsf{out}_{\mathrm{hi}}$ is written only by the full 16-element permutation).

5.7 EXTENSION table

5.7.1 Trace layout

Each extension field operation occupies N consecutive rows, where N is the length parameter of the `EXTENSION_OP` instruction (the number of element pairs in the dot product / sum / eq operation, see Section 4.7.6): one row per element pair, with the column len counting down from N on the first row to 1 on the last. Within a group, rows are ordered from first element to last, but the acc column accumulates backward: the first row holds the full result, the last row holds only the final element's value. Padding rows have all operation flags set to 0 and $\mathsf{flag\_start} = 1$, $\mathsf{len} = 1$.

flag_be	flag_start	flag_add	flag_dot_product	flag_eq	len	idx_A	idx_B	idx_R	v_A (d columns)	v_B (d columns)	res (d columns)	acc (d columns)	aux_2
DOT_PRODUCT, base × extension, $N=4$ ($\nu_A=90$, $\nu_B=211$, $\nu_C=74$): $\mathsf{res} = \sum_{i=0}^{3} \boldsymbol{m}[90+i] \cdot \boldsymbol{m}[211+5i..216+5i]$													
1	1	0	1	0	4	90	211	74	$\mathbf{m}[90..95]$	$\mathbf{m}[211..216]$	$\mathbf{m}[74..79]$	$e_0+e_1+e_2+e_3$	276
1	0	0	1	0	3	91	216	74	$\mathbf{m}[91..96]$	$\mathbf{m}[216..221]$	$\mathbf{m}[74..79]$	$e_1+e_2+e_3$	212
1	0	0	1	0	2	92	221	74	$\mathbf{m}[92..97]$	$\mathbf{m}[221..226]$	$\mathbf{m}[74..79]$	e_2+e_3	148
1	0	0	1	0	1	93	226	74	$\mathbf{m}[93..98]$	$\mathbf{m}[226..231]$	$\mathbf{m}[74..79]$	e_3	84
ADD, extension × extension, $N=3$ ($\nu_A=400$, $\nu_B=500$, $\nu_C=300$): $\mathsf{res} = \sum_{i=0}^{2} (\boldsymbol{m}[400+5i..405+5i] + \boldsymbol{m}[500+5i..505+5i])$													
0	1	1	0	0	3	400	500	300	$\mathbf{m}[400..405]$	$\mathbf{m}[500..505]$	$\mathbf{m}[300..305]$	$e_0+e_1+e_2$	200
0	0	1	0	0	2	405	505	300	$\mathbf{m}[405..410]$	$\mathbf{m}[505..510]$	$\mathbf{m}[300..305]$	e_1+e_2	136
0	0	1	0	0	1	410	510	300	$\mathbf{m}[410..415]$	$\mathbf{m}[510..515]$	$\mathbf{m}[300..305]$	e_2	72
EQ, extension × extension, $N=3$ ($\nu_A=600$, $\nu_B=700$, $\nu_C=800$): $\mathsf{res} = \prod_{i=0}^{2} [a_i b_i + (1-a_i)(1-b_i)]$													
0	1	0	0	1	3	600	700	800	$\mathbf{m}[600..605]$	$\mathbf{m}[700..705]$	$\mathbf{m}[800..805]$	$e_0 \cdot e_1 \cdot e_2$	224
0	0	0	0	1	2	605	705	800	$\mathbf{m}[605..610]$	$\mathbf{m}[705..710]$	$\mathbf{m}[800..805]$	$e_1 \cdot e_2$	160
0	0	0	0	1	1	610	710	800	$\mathbf{m}[610..615]$	$\mathbf{m}[710..715]$	$\mathbf{m}[800..805]$	e_2	96
Padding row													
0	1	0	0	0	1	0	0	0	$\mathbf{m}[0..5]$	$\mathbf{m}[0..5]$	$\mathbf{m}[0..5]$	0	64

5.7.2 Columns

There are 29 columns. Extension field values are represented as d consecutive columns.

- $\mathsf{flag\_be} \in \{0, 1\}$: mode flag (1: base $\times$ extension, 0: extension $\times$ extension)
- $\mathsf{flag\_start} \in \{0, 1\}$: group boundary (1 on first row of each group and on padding, 0 elsewhere)
- $\mathsf{flag\_add}$, $\mathsf{flag\_dot\_product}$, $\mathsf{flag\_eq} \in \{0, 1\}$: operation selectors (exactly one is 1 on active rows, all 0 on padding)
- len: countdown from N to 1 within each group (1 on padding)
- idx_A, idx_B, idx_R: memory addresses of the two operands and of the result
- $v_A = (v_A[0], \ldots, v_A[4])$ (d columns): the d base field values claimed to live at memory addresses $\mathsf{idx}_A, \ldots, \mathsf{idx}_A + 4$ (enforced by the memory pulls below). In extension $\times$ extension mode, v_A is interpreted as the extension field element $\mathsf{embed}(v_A)$; in base $\times$ extension mode, only $v_A[0]$ is used (as a base field scalar) and the remaining coordinates $v_A[1..d]$ are unconstrained.
- $v_B = (v_B[0], \ldots, v_B[4])$ (d columns): same as v_A at addresses $\mathsf{idx}_B, \ldots, \mathsf{idx}_B + 4$; always interpreted as the extension field element $\mathsf{embed}(v_B)$.
- $\mathsf{res} = (\mathsf{res}[0], \ldots, \mathsf{res}[4])$ (d columns): the d base field values of the output extension field element, claimed to live at addresses $\mathsf{idx}_R, \ldots, \mathsf{idx}_R + 4$. Constant on all rows of a group.
- $\mathsf{acc} = (\mathsf{acc}[0], \ldots, \mathsf{acc}[4])$ (d columns): running accumulator (over $\mathbb{F}_q$) of the per-element contributions $e_\bullet$ defined in the AIR constraints below; on $\mathsf{flag\_start}$ rows it equals res.

Virtual (non-committed) columns: $\mathsf{multiplicity} = \mathsf{flag\_start} \cdot (\mathsf{flag\_add} + \mathsf{flag\_dot\_product} + \mathsf{flag\_eq})$ and $\mathsf{aux}_2 = 4 \cdot \mathsf{flag\_be} + 8 \cdot \mathsf{flag\_add} + 16 \cdot \mathsf{flag\_dot\_product} + 32 \cdot \mathsf{flag\_eq} + 64 \cdot \mathsf{len}$

5.7.3 Bus interactions

Memory pulls. For each operand $(\mathsf{idx}, s) \in \{(\mathsf{idx}_A, v_A), (\mathsf{idx}_B, v_B), (\mathsf{idx}_R, \mathsf{res})\}$ and each $k \in [0, d)$:

$$\textsc{Pull}\big(\mathsf{idx} + k,\ s[k],\ 0_{13},\ \mathsf{memory\_sep}\big) \quad \text{with multiplicity } 1.$$

Precompile pull.

$$\textsc{Pull}\big(\mathsf{idx}_A,\ \mathsf{idx}_B,\ \mathsf{idx}_R,\ 0_{12},\ \mathsf{aux}_2\big) \quad \text{with multiplicity } \mathsf{multiplicity},$$

The domain separator aux_2 binds the mode and length to the bus data: since $\mathsf{flag\_be}, \mathsf{flag\_add}, \mathsf{flag\_dot\_product}, \mathsf{flag\_eq}$ are boolean and the length is $\leq 2^{22}$ (Lemma 3), the encoding is injective, so every aux_2 sent by the execution table is faithfully received here.

5.7.4 AIR constraints

We use AIR constraints of degree 6.

Let $\tilde{v}_A = (v_A[0],\ (1 - \mathsf{flag\_be}) \cdot v_A[1],\ \ldots,\ (1 - \mathsf{flag\_be}) \cdot v_A[4])$, and define the per-element (virtual) results

$$e_{\text{add}} = \tilde{v}_A + v_B, \qquad e_{\text{dot\_product}} = \tilde{v}_A \cdot v_B, \qquad e_{\text{eq}} = \tilde{v}_A \cdot v_B + (1 - \tilde{v}_A)(1 - v_B),$$

where sums and products are in $\mathbb{F}_q = \mathbb{F}_p[X]/(X^5 + X^2 - 1)$ (Section 4.1). Each $e_\bullet$ is therefore a d-tuple of base field expressions (one per coordinate), and each coordinate-valued constraint below stands for d base field constraints.

Let $\mathsf{acc\_tail} = \text{next}(\mathsf{acc}) \cdot (1 - \text{next}(\mathsf{flag\_start}))$.

Boolean flags. Each of $\mathsf{flag\_be}, \mathsf{flag\_start}, \mathsf{flag\_add}, \mathsf{flag\_dot\_product}, \mathsf{flag\_eq}$ is constrained to be boolean:

- **(2)** $f \cdot (1 - f) = 0$ for $f \in \{\mathsf{flag\_be}, \mathsf{flag\_start}, \mathsf{flag\_add}, \mathsf{flag\_dot\_product}, \mathsf{flag\_eq}\}$.

Per-row accumulation and result. The running accumulator acc propagates the per-element contribution backwards:

- $\mathsf{flag\_add} \cdot (\mathsf{acc} - e_{\text{add}} - \mathsf{acc\_tail}) = 0$ (ADD: additive accumulation)
- $\mathsf{flag\_dot\_product} \cdot (\mathsf{acc} - e_{\text{dot\_product}} - \mathsf{acc\_tail}) = 0$ (DOT_PRODUCT: additive accumulation)
- $\mathsf{flag\_eq} \cdot (\mathsf{acc} - e_{\text{eq}} \cdot (\mathsf{acc\_tail} + \text{next}(\mathsf{flag\_start}))) = 0$ (EQ: multiplicative accumulation)
- $\mathsf{flag\_start} \cdot (\mathsf{acc} - \mathsf{res}) = 0$ (on group-start rows, $\mathsf{acc} = \mathsf{res}$)

Intra-group consistency. Within a group (i.e. when the next row is not a group start), the mode/operation flags are constant, len counts down by 1, and the input pointers advance by their natural stride (1 base field cell for idx_A in base $\times$ extension mode, d cells otherwise):

- **(3)** $(1 - \text{next}(\mathsf{flag\_start})) \cdot (\mathsf{len} - \text{next}(\mathsf{len}) - 1) = 0$ (len countdown)
- $(1 - \text{next}(\mathsf{flag\_start})) \cdot (f - \text{next}(f)) = 0$ for $f \in \{\mathsf{flag\_be}, \mathsf{flag\_add}, \mathsf{flag\_dot\_product}, \mathsf{flag\_eq}\}$ (mode/operation constant)
- $(1 - \text{next}(\mathsf{flag\_start})) \cdot (\text{next}(\mathsf{idx}_A) - \mathsf{idx}_A - (\mathsf{flag\_be} + (1 - \mathsf{flag\_be}) \cdot d)) = 0$ (idx_A stride)
- $(1 - \text{next}(\mathsf{flag\_start})) \cdot (\text{next}(\mathsf{idx}_B) - \mathsf{idx}_B - d) = 0$ (idx_B stride)

Group boundary. When the next row starts a new group, the current group must have reached its last element:

- **(4)** $\text{next}(\mathsf{flag\_start}) \cdot (\mathsf{len} - 1) = 0$ ($\mathsf{len} = 1$ on the last row of a group)

Lemma 3. On every row r of the extension table, $\mathsf{len}_r \leq H - r$ (as an integer in $\{0, \ldots, p-1\}$), where $H \leq 2^{22}$ is the table height. In particular, $\mathsf{len}_r \leq H$.

Proof. By backward induction on r, from $r = H - 1$ down to $r = 0$.

Base case ($r = H - 1$): On the wrapping pair (row $H-1$ paired with itself, see Section 3.2.5), $\text{next}(x) = x$ for every column x. Constraint (3) becomes $(1 - \mathsf{flag\_start}) \cdot (\mathsf{len} - \mathsf{len} - 1) = -(1 - \mathsf{flag\_start}) = 0$, forcing $\mathsf{flag\_start}_{H-1} = 1$ (which is itself in $\{0, 1\}$ by (2) applied to $f = \mathsf{flag\_start}$). Then (4) becomes $1 \cdot (\mathsf{len} - 1) = 0$, giving $\mathsf{len}_{H-1} = 1 = H - (H - 1)$.

Inductive step ($r < H - 1$): Assume $\mathsf{len}_s \leq H - s$ for all $s > r$. The pair (row r, row $r+1$) gives two cases depending on $\mathsf{flag\_start}_{r+1} \in \{0, 1\}$ (by (2) applied to $f = \mathsf{flag\_start}$):

- If $\mathsf{flag\_start}_{r+1} = 1$: (4) gives $1 \cdot (\mathsf{len}_r - 1) = 0$, so $\mathsf{len}_r = 1 \leq H - r$.
- If $\mathsf{flag\_start}_{r+1} = 0$: (3) gives $(1 - 0) \cdot (\mathsf{len}_r - \mathsf{len}_{r+1} - 1) = 0$, so $\mathsf{len}_r = \mathsf{len}_{r+1} + 1$ in $\mathbb{F}_p$. By induction, $\mathsf{len}_{r+1} \leq H - (r+1)$, so $\mathsf{len}_{r+1} + 1 \leq H - r$. Since $H - r \leq H \leq 2^{22} < p$, the addition does not overflow modulo p, so $\mathsf{len}_r = \mathsf{len}_{r+1} + 1 \leq H - r$ as integers.

□

TODO: Verify and (formally) prove soundness.

5.8 End-to-end protocol

Verification parameters (known to both prover and verifier).

- bytecode: the bytecode of the program being executed (Section 5.5.2), of size $2^{h_{\text{BYTECODE}}}$, assumed well-formed (Section 4.4).
- public_input: the initial public part of the memory, occupying cells $[0, \ell_{\text{PUBLIC}})$ with $\ell_{\text{PUBLIC}} = 8$.

1. **Dimensions.** The prover sends $(\mathsf{log\_inv\_rate}, h_{\text{MEMORY}}, h_{\text{EXEC}}, h_{\text{POSEIDON}}, h_{\text{EXTENSION}})$.

   The verifier checks: $\mathsf{log\_inv\_rate}^{\min} \le \mathsf{log\_inv\_rate} \le \mathsf{log\_inv\_rate}^{\max}$ (i.e. $1 \le \mathsf{log\_inv\_rate} \le 4$); $h^{\min} \le h_T \le h_T^{\max}$ for each $T \in \{\text{EXEC}, \text{POSEIDON}, \text{EXTENSION}\}$ (Section 5.2); $h_{\text{MEMORY}}^{\min} \le h_{\text{MEMORY}} \le h_{\text{MEMORY}}^{\max}$; $h_{\text{BYTECODE}} \ge h^{\min}$; and $h_{\text{MEMORY}} \ge \max(h_{\text{BYTECODE}}, h_{\text{EXEC}}, h_{\text{POSEIDON}}, h_{\text{EXTENSION}})$.

2. **Stacked commitment.** The prover commits via WHIR [3] to a single multilinear polynomial stacking (see Section 5.1) the memory $\mathbf{m}$, the access counts memory_acc and bytecode_acc, and all committed columns of the three tables.

3. **Logup.**

   (a) The verifier samples $\gamma \in \mathbb{F}_q$ and $\vec{\beta} \in \mathbb{F}_q^{\ell}$ (bus-tuple hashing seeds, $\ell = \log_2 \mathsf{bus\_width}$), used for Equation (1).

   (b) Run the GKR protocol of Section 5.3 to compute the sum of quotients from Equation (1). The sum contains terms from: the memory section ($2^{h_{\text{MEMORY}}}$ entries) and the bytecode section ($\max(2^{h_{\text{BYTECODE}}}, 2^{h_{\text{EXEC}}})$ entries, padded if $h_{\text{BYTECODE}} < h_{\text{EXEC}}$; see Section 5.3.2), then, for each table T sorted by descending height, one block of 2^{h_T} entries per bus interaction of T. Write h_{GKR} for the log2-ceiling of the resulting total length.
   The protocol yields a point $\vec{r}_{\text{GKR}} \in \mathbb{F}_q^{h_{\text{GKR}}}$ and two scalar claims $(e_{\text{num}}, e_{\text{den}})$ on the numerator and denominator MLEs at $\vec{r}_{\text{GKR}}$.

   (c) The prover sends, at the appropriate suffixes of $\vec{r}_{\text{GKR}}$:

   - the evaluations of memory_acc and $\mathbf{m}$ for the memory section;
   - the evaluation of bytecode_acc for the bytecode section;
   - for each table T and each bus interaction of T, the per-column evaluations needed to reconstruct that bus's numerator/denominator contribution.

   The verifier recombines these evaluations into a claimed total numerator and denominator and checks they equal $(e_{\text{num}}, e_{\text{den}})$.

4. **AIR.**

   (a) The verifier samples $\mu \in \mathbb{F}_q$. Consecutive powers $1, \mu, \mu^2, \ldots$ are used to take a random linear combination of *all* verification constraints across the three tables.

   (b) Prover and verifier run the back-loaded batched zerocheck of Section 5.4.3, merging the per-table constraint polynomials into a single sumcheck of $h_{\text{MAX}} := \max_T h_T$ rounds, *crucially reusing the suffix of $\vec{r}_{\text{GKR}}$ of length h_{MAX} from the logup step as the zerocheck random challenge (no fresh point is sampled).* TODO explain why do this, and TODO explain why it's sound. Let $\vec{r}_{\text{AIR}} \in \mathbb{F}_q^{h_{\text{MAX}}}$ denote the sumcheck challenges.

   (c) For each table T, the prover sends the evaluations of all of T's committed columns and shifted columns at the appropriate suffix of $\vec{r}_{\text{AIR}}$. The verifier evaluates T's AIR-constraint polynomial at these column evaluations, which enables to conclude the batched-zerocheck.

5. **Public-memory consistency.** The verifier samples $\vec{r}_{\text{PUB}} \in \mathbb{F}_q^{\log_2 \ell_{\text{PUBLIC}}}$ and evaluates public_input as a multilinear polynomial at $\vec{r}_{\text{PUB}}$, obtaining $v_{\text{PUB}} \in \mathbb{F}_q$. The pair $(\vec{r}_{\text{PUB}}, v_{\text{PUB}})$ is queued as an opening claim on the initial ℓ_{PUBLIC}-cell prefix of the committed memory polynomial (handled by WHIR below).

6. **Boundary pc.** To ensure a valid initial / final program counter, we add two opening claims on the **pc** column of the EXECUTION table: $\hat{\mathbf{pc}}(0, \dots, 0) = 0$ and $\hat{\mathbf{pc}}(1, \dots, 1) = 2^{h_{\text{BYTECODE}}} - 1$ (handled by WHIR bellow).

7. **WHIR opening.** The prover and verifier run a single WHIR verification on the stacked commitment of step 2 against the list of evaluation claims (including shifted ones) gathered so far.

5.9 Sponge

A slice v of field elements, of arbitrary length $L \geq 1$, is hashed into a digest $\mathsf{H}(v) \in \mathbb{F}_p^{\ell_{\text{DIGEST}}}$ by an *overwrite sponge* over the permutation $\texttt{POSEIDON}^\pi$, whose state splits into a *rate* of ℓ_{DIGEST} elements and a *capacity* of equal size 8. Writing a state as $(c \,\|\, r)$ with capacity c and rate r, split v into $n = \lceil L/\ell_{\text{DIGEST}} \rceil$ blocks $B_0, \dots, B_{n-1}$ of ℓ_{DIGEST} elements (zero-padding the last). Each block *overwrites* the rate before the state is permuted, starting from a capacity that carries the length:

$$c_0 = (L, 0, \dots, 0), \qquad (c_{i+1} \,\|\, r_{i+1}) = \texttt{POSEIDON}^\pi(c_i \,\|\, B_i) \quad (0 \leq i < n), \qquad \mathsf{H}(v) = r_n.$$

5.10 Fiat-Shamir

In order to turn the interactive protocol described in Section 5.8 into a non-interactive one, we apply the BCS transformation [12]. In particular, we use the Fiat-Shamir transformation implemented via a duplex sponge [13] (using the $\texttt{POSEIDON}^\pi$ permutation over a state of 16 KoalaBear field elements).

6 ISA programming

This section describes *conventions* for writing programs using the ISA of Section 4.7. None of it is hard-coded into the VM or enforced by the snark.

6.1 Hints

A *hint* is the use of non-determinism: the prover supplies a value, the program then verifies it, in order to replace an expensive computation by a cheap check.

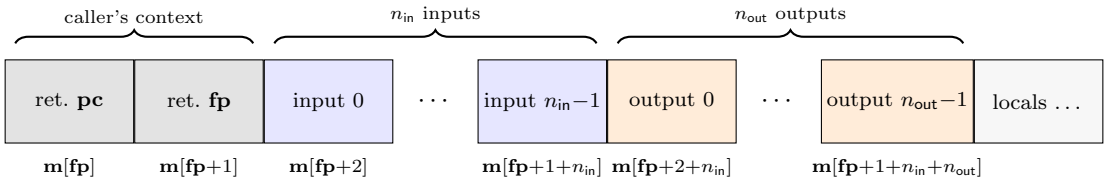
Example 6.1. Computing an inverse $y := x^{-1} \in \mathbb{F}_p$. y is hinted to some memory cell and the program performs the single check $x \cdot y = 1$: one MUL.

6.2 Functions

A *function* is a piece of bytecode, associated to:

- a starting **pc**
- a number of inputs n_{in}
- a number of outputs n_{out}
- a frame size $s \geq 2 + n_{\text{in}} + n_{\text{out}}$ (in memory)

Every call operates on a fresh contiguous block of s memory cells: its *frame*, pointed to by **fp** during execution. By convention, this frame contains the following data:



Call. Say the caller is at $\mathbf{pc} = \mathbf{pc}_{\text{caller}}$, $\mathbf{fp} = \mathbf{fp}_{\text{caller}}$, about to invoke a function f with entry point $\mathbf{pc}_f$ and inputs $a_0, \dots, a_{n_{\text{in}}-1}$ (either constants or memory cells). It obtains a hinted frame pointer $\mathbf{fp}_{\text{callee}}$ (supposedly pointing to a fresh interval of size s in memory) and executes:

- $\mathbf{m}[\mathbf{fp}_{\text{callee}}] \leftarrow \mathbf{pc}_{\text{ret}}$, where $\mathbf{pc}_{\text{ret}} = \mathbf{pc}_{\text{caller}} + 2 + n_{\mathsf{in}}$ is the address of the instruction following the JUMP below;
- $\mathbf{m}[\mathbf{fp}_{\text{callee}} + 1] \leftarrow \mathbf{fp}_{\text{caller}}$;
- $\mathbf{m}[\mathbf{fp}_{\text{callee}} + 2 + k] \leftarrow a_k$ for $k = 0, \ldots, n_{\mathsf{in}} - 1$;
- JUMP with $\mathbf{pc} \leftarrow \mathbf{pc}_f$ and $\mathbf{fp} \leftarrow \mathbf{fp}_{\text{callee}}$.

Return. Before returning, the callee has written its n_{out} outputs $r_0, \ldots, r_{n_{\mathsf{out}}-1}$ to $\mathbf{m}[\mathbf{fp}+2+n_{\mathsf{in}}+k] \leftarrow r_k$ for $k = 0, \ldots, n_{\mathsf{out}} - 1$. It then executes the JUMP $\mathbf{pc} \leftarrow \mathbf{m}[\mathbf{fp}]$, $\mathbf{fp} \leftarrow \mathbf{m}[\mathbf{fp}+1]$, which reads back $\mathbf{pc}_{\text{ret}}$ and $\mathbf{fp}_{\text{caller}}$. Execution resumes at $\mathbf{pc}_{\text{ret}}$ in the caller's original frame.

No allocation pointer. Cairo [2] keeps a register **ap** pointing to the next free memory cell. leanVM does not: the write-once memory (Section 4.3) prevents any frame from overwriting previously-written data. The following convention is thus crucial: *a function's internal logic should only depend on its inputs, not on where its frame happens to land*, so that the value of $\mathbf{fp}_{\text{callee}}$ does not impact its execution.

The initial fp is likewise unconstrained. For the same reason, the protocol does not pin the first row's **fp** (the boundary checks of Section 5.8 fix only the initial and final **pc**): program logic is not expected to depend on the exact value of **fp**, only on the memory values indexed relative to it.

The ISA makes it possible to use values of **fp** outside the range $[0, 2^{h_{\text{MEMORY}}})$. As long as every memory access of the form $\mathbf{m}[\mathbf{fp} + x]$ (for some offset $x \in \mathbb{F}_p$) is in range, i.e. $\mathbf{fp} + x < 2^{h_{\text{MEMORY}}}$, the execution is valid. Yet, reasoning about such out-of-range values of **fp** is tricky, and we thus recommend the following two programming conventions, that ensure **fp** always stays in range $[0, 2^{h_{\text{MEMORY}}})$:

- The first instruction of the program should be $\texttt{ADD}[\texttt{imm}, 0, \ \texttt{imm}, 0, \ \texttt{mem}, 0]$, of semantics $\mathbf{m}[\mathbf{fp}] = 0$: the memory access at address **fp** forces the initial $\mathbf{fp} < 2^{h_{\text{MEMORY}}}$ (Lemma 2).
- Whenever a JUMP updates **fp**, the program should guarantee that the new value is in range. The calling convention above does it for free:
  - On a call, $\mathbf{fp} \leftarrow \mathbf{m}[\mathbf{fp} + x]$, where $\mathbf{m}[\mathbf{fp} + x] = \mathbf{fp}_{\text{callee}}$ is hinted, hence arbitrary. But the callee frame was initialized with the DEREF $\mathbf{m}[\mathbf{m}[\mathbf{fp} + x] + 0] \leftarrow \mathbf{pc}_{\text{ret}}$, whose memory access at address $\mathbf{m}[\mathbf{fp} + x]$ forces $\mathbf{m}[\mathbf{fp} + x] < 2^{h_{\text{MEMORY}}}$ (Lemma 2).
  - On a return, $\mathbf{fp} \leftarrow \mathbf{m}[\mathbf{fp} + 1]$ reads back what the caller wrote there, i.e. its own **fp**, in range by induction.

6.3 Loops

We suggest to unroll loops when the number of iterations is low, and known at compile time.

The remaining loops are transformed into recursive functions (by a compiler):

Example 6.2. Summing the n entries of an array a:

loop form

```
sum(a, n):
    s = 0
    for i in [0, n):
        s = s + a[i]
    return s
```

recursive form

```
sum(a, n):
    return sum_rec(a, n, 0, 0)

sum_rec(a, n, i, s):
    if i == n:
        return s
    return sum_rec(a, n, i + 1, s + a[i])
```

6.4 Range checks

It's possible to check that a memory cell is smaller than t (for $t \leq 2^{16}$) in 3 cycles, where t is either a constant or a memory cell [1].

[1] This idea was pointed out by D. Khovratovich, and is an unplanned use of the DEREF instruction

In case t is a memory cell, the program should crucially enforce $t \leq 2^{16}$ (potentially using, again, a range check).

We denote by $\mathbf{m}[\mathbf{fp} + x]$ the memory cell for which we want to ensure $\mathbf{m}[\mathbf{fp} + x] < t$. We also denote by $\mathbf{m}[\mathbf{fp} + i]$, $\mathbf{m}[\mathbf{fp} + j]$ and $\mathbf{m}[\mathbf{fp} + k]$ 3 auxiliary memory cells (that have not been used yet).

1. $\mathbf{m}[\mathbf{m}[\mathbf{fp} + x]] = \mathbf{m}[\mathbf{fp} + i]$ (using DEREF, this ensures $\mathbf{m}[\mathbf{fp} + x] < M$, the memory size, by Lemma 2)
2. $\mathbf{m}[\mathbf{fp} + x] + \mathbf{m}[\mathbf{fp} + j] =$ (t-1) (using ADD, this computes $\mathbf{m}[\mathbf{fp}+\mathrm{j}] = t - 1 - \mathbf{m}[\mathbf{fp} + x]$)
3. $\mathbf{m}[\mathbf{m}[\mathbf{fp} + j]] = \mathbf{m}[\mathbf{fp} + k]$ (using DEREF, this ensures $t - 1 - \mathbf{m}[\mathbf{fp} + x] < M$)

Given $t \leq 2^{16} \leq 2^{h_{\text{MEMORY}}}$, $\mathbf{m}[\mathbf{fp} + x] < 2^{h_{\text{MEMORY}}}$, $t - 1 - \mathbf{m}[\mathbf{fp} + x] < 2^{h_{\text{MEMORY}}}$, and $2^{h_{\text{MEMORY}}} \leq 2^{26} < p/2$, we have: $\mathbf{m}[\mathbf{fp} + x] < t$.

Note: From the point of view of the prover running the program: filling the values of $\mathbf{m}[\mathbf{fp} + i]$ and $\mathbf{m}[\mathbf{fp} + k]$ must be done at end of execution.

Example 6.3. Let's say we want to write a function with 2 arguments $x = \mathbf{m}[\mathbf{fp} + 2]$ and $y = \mathbf{m}[\mathbf{fp} + 3]$ ($\mathbf{m}[\mathbf{fp} + 0]$ and $\mathbf{m}[\mathbf{fp} + 1]$ are used, by convention, to store the caller's **pc** and **fp**, to return to the previous context at the end of the function), which perform the following:

1. assert(x < 10)
2. z := x*y + 100
3. assert(z < 1000)

Which can be compiled to:

1. $\mathbf{m}[\mathbf{fp} + 4] = \mathbf{m}[\mathbf{m}[\mathbf{fp} + 2]]$ // check that x is "small"
2. $\mathbf{m}[\mathbf{fp} + 2] + \mathbf{m}[\mathbf{fp} + 5] = 9$ // compute $9 - x$
3. $\mathbf{m}[\mathbf{fp} + 6] = \mathbf{m}[\mathbf{m}[\mathbf{fp} + 5]]$ // check that $9 - x$ is "small"
4. $\mathbf{m}[\mathbf{fp} + 7] = \mathbf{m}[\mathbf{fp} + 2] * \mathbf{m}[\mathbf{fp} + 3]$ // compute $x.y$
5. $\mathbf{m}[\mathbf{fp} + 8] = \mathbf{m}[\mathbf{fp} + 7] + 100$ // compute $z = x.y + 100$
6. $\mathbf{m}[\mathbf{fp} + 9] = \mathbf{m}[\mathbf{m}[\mathbf{fp} + 8]]$ // check that z is "small"
7. $\mathbf{m}[\mathbf{fp} + 8] + \mathbf{m}[\mathbf{fp} + 10] = 999$ // compute $999 - z$
8. $\mathbf{m}[\mathbf{fp} + 11] = \mathbf{m}[\mathbf{m}[\mathbf{fp} + 10]]$ // check that $999 - z$ is "small"
9. JUMP with $\text{next}(\mathbf{pc}) = \mathbf{m}[\mathbf{fp} + 0]$, $\text{next}(\mathbf{fp}) = \mathbf{m}[\mathbf{fp} + 1]$, condition = 1 // return

6.5 Switch statements

A switch dispatches to one of k different code blocks based on a runtime value x known to satisfy $0 \leq x < k$ (if x came from a hint, it must be range-checked first; see Section 6.4).

Implementation: place the k blocks at regular intervals in the bytecode, all of the same length b (the smaller blocks are padded with unreachable instructions), starting at $\mathbf{pc} = a$ (the block for $x = 0$). The dispatch is then a single JUMP to $\mathbf{pc} = a + b \cdot x$.

Example 6.4. Computing the x-th decimal digit of $\pi = 3.1415\ldots$ for $x \in \{0, 1, 2, 3\}$:

$$r = \begin{cases} 3 & x = 0 \\ 1 & x = 1 \\ 4 & x = 2 \\ 1 & x = 3 \end{cases}$$

```
 ;; dispatch (a = 100, b = 2, x = m[fp+3])
 97: MUL[imm, 2,    mem, 4,   mem, 3]      ; m[fp+4] = 2 * x
 98: ADD[imm, 100,  mem, 5,   mem, 4]      ; m[fp+5] = 100 + 2*x
 99: JUMP[imm, 1,   mem, 5,   fp_rel, 0]   ; pc <- m[fp+5]

 ;; 4 blocks, each of length b = 2, laid out at regular intervals
100: ADD[imm, 3,    mem, 6,   imm, 0]      ; res = 3
101: JUMP[imm, 1,   imm, 108, fp_rel, 0]   ; pc <- 108

102: ADD[imm, 1,    mem, 6,   imm, 0]      ; res = 1
103: JUMP[imm, 1,   imm, 108, fp_rel, 0]   ; pc <- 108

104: ADD[imm, 4,    mem, 6,   imm, 0]      ; res = 4
105: JUMP[imm, 1,   imm, 108, fp_rel, 0]   ; pc <- 108

106: ADD[imm, 1,    mem, 6,   imm, 0]      ; res = 1
107: JUMP[imm, 1,   imm, 108, fp_rel, 0]   ; pc <- 108

108: ...                                                  ;; merge
```

6.6 Compiling from a high-level zkDSL

A high-level zkDSL (with mutable variables, loops, and conditionals) can be compiled to the ISA, by systematically translating mutation into *Static Single-Assignment* (SSA) form, and every loop by a recursive function (Section 6.3).

Example 6.5. *source*

```
def func(n):
    x: Mut = 1
    x += 2
    x = x * 4
    for i in range(0, n):
        x += i
    return x
```

after an intial compilation pass

```
def func(n):
    x1 = 1
    x2 = x1 + 2
    x3 = x2 * 4
    x_buff = Array(n + 1)
    x_buff[0] = x3
    loop(0, n, x_buff)
    return x_buff[n]

def loop(i, n, x_buff):
    if i == n:
        return
    x_buff[i+1] = x_buff[i] + i
    loop(i + 1, n, x_buff)
```

Here $\mathsf{Array}(k)$ is a hint: a prover-supplied pointer to a fresh region of k memory cells.

7 Signature aggregation

XMSS [7] is a stateful hash-based signature scheme: each signature is bound to a nonce s. Aggregated via leanVM, XMSS is a promising candidate as a Post-Quantum replacement for BLS [6] at the consensus layer of Ethereum.

Statement we prove. Fix a slot s, a message msg and a list of public keys $\mathsf{pk}_1, \ldots, \mathsf{pk}_K$. In Ethereum Consensus' setting, s is the current consensus slot and msg is the block hash. The aggregation proof attests knowledge soundness of:

$$\exists\, \sigma_1, \ldots, \sigma_K \;\text{ such that }\; \mathsf{XMSS.Verify}(\mathsf{pk}_k, \mathsf{msg}, s, \sigma_k) = \text{TRUE} \quad \text{for every } k \in [1, K].$$

The proof's public input is $(s, \mathsf{msg}, \mathsf{pk}_1, \ldots, \mathsf{pk}_K)$; the individual signatures are part of the witness.

We need to support recursive aggregation: being able to further aggregate a list of previously aggregated signatures.

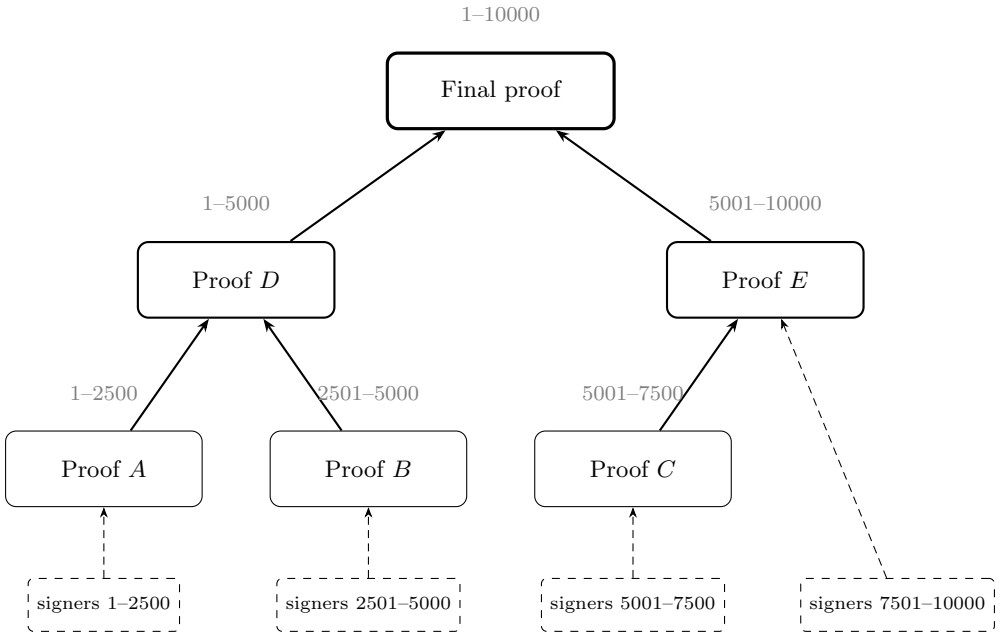


Figure 2: Example of recursive aggregation of 10000 XMSS, sharing a common message. (Note: overlapping sets of signers are possible.)

7.1 Recursive Aggregation

The recursive aggregation program (see Figure 2) attests that a set of public keys have valid signatures for a given message. It partitions the signers into $n_{\text{rec}} + 1$ sources: one **direct source** whose XMSS signatures are verified explicitly, and $n_{\text{rec}} \geq 0$ **recursive sources**, each accompanied by a sub-proof that is recursively verified.

```
def aggregate(slot, msg, pks):       # msg:message, pks: public keys

    # the prover partitions pks into 1 direct subset + n_rec recursive subsets
    direct = hint()                  # subset of pks to verify directly
    n_rec  = hint()                  # number of recursive groups
    groups = hint()                  # groups[j] = subset of pks for j = 0..n_rec-1

    for pk in direct:
        sig = hint()
        assert XMSS.Verify(pk, msg, slot, sig)

    for j in range(n_rec):
        sub_proof = hint()
        assert leanVM.Verify(
            bytecode      = THIS_PROGRAM,  # self-dependency, see section below
            public_input  = (slot, msg, groups[j]),
            proof         = sub_proof,
        )

    # every pk in pks must appear in direct or in some groups[j]
    assert_full_coverage(pks, direct, groups)
```

7.1.1 Bytecode evaluation claims

A key subtlety arises from recursion. Every leanVM proof is tied to a specific *bytecode* — the program that was executed, represented as a multilinear polynomial. In our case, there is a single program (that contains both the signature verification and recursion algorithms). The leanVM verification algorithm requires evaluating the bytecode polynomial at a random point.

When the recursive aggregation program verifies a sub-proof, it runs the leanVM verifier *inside* the program. This verification produces a bytecode evaluation claim. For efficiency, rather than checking it inside the program (with a PCS), the claim is forwarded to the public input, to be checked externally.

However, each sub-proof is potentially itself a recursive aggregation, and may have already forwarded its own bytecode claim in the same way. So for each of the n_{rec} sub-proofs, **two** bytecode claims arise:

1. **Inner-public-memory claim**: The bytecode claim that the sub-proof forwarded from its own recursive verifications (read from the sub-proof's public input).
2. **Inner-proof claim**: The new bytecode claim produced by verifying the sub-proof itself.

After verifying all n_{rec} sub-proofs, a fresh Fiat–Shamir transcript is initialized and fed with all $2n_{\text{rec}}$ evaluation points and claimed values. This transcript produces a random linear combination challenge, and the $2n_{\text{rec}}$ claims are then batched into a single one via sumcheck, yielding a reduced claim. This reduced claim is written to the public input for the next level of recursion.

When $n_{\text{rec}} = 0$ (no recursion), the bytecode claim in the public input is set to zero.

7.1.2 Signer partitioning

In the following, we assume the message being signed is common to all signers (typically the case at Ethereum's consensus layer), but the scheme can be naturally adapted to distinct messages.

Consider an aggregation of n_{sigs} (distinct) signatures. The n_{sigs} (distinct) public keys are part of the "public input" in memory. The aggregation program must ensure that each public key is verified by at least one of the $1 + n_{\text{rec}}$ sources (duplications allowed: some public keys may be verified by multiple sources).

Let n_{dup} be the total number of duplicated public keys across sources, counted with multiplicity. To ensure valid partitioning, we use the following algorithm:

- Initialize a counter $c \leftarrow 0$.
- Initialize a (write-once) array B of size $n_{\text{total}} = n_{\text{sigs}} + n_{\text{dup}}$.
- For each source s, for each signature index $i \in [0, n_{\text{total}})$ verified by s, write $B[i] \leftarrow c$ and increment c.
- At the end, assert $c = n_{\text{total}}$.

7.2 Public-input layout

The aggregation program supports two flavors, selected by a leading flag:

- **Single-message** ($\mathsf{flag}_{\text{SINGLE}} = 1$): a single (msg, s) aggregation, where every public key in the list signed the same message msg at the same slot s. It combines any mix of raw XMSS signatures (verified directly) and child single-message aggregates (verified recursively, Section 7.1).
- **Multi-message** ($\mathsf{flag}_{\text{MULTI}} = 0$): a bundle of n independent single-message aggregates, each with its own (msg, s) and pubkey set.

The public input is the hash of a field-element buffer (via the sponge of **??**). The buffer is defined as follows:

Offset	Size	Contents
0	ℓ_{DIGEST}	[flag, count, $0, \ldots, 0$]
ℓ_{DIGEST}	c	bytecode evaluation claim (Section 7.1.1), zero-padded to whole digests
$\ell_{\text{DIGEST}} + c$	ℓ_{DIGEST}	bytecode-hash domain separator
single-message tail (fixed, 4 digests):		
$2\ell_{\text{DIGEST}} + c$	ℓ_{DIGEST}	hash of all aggregated public keys
$3\ell_{\text{DIGEST}} + c$	ℓ_{DIGEST}	message msg
$4\ell_{\text{DIGEST}} + c$	ℓ_{DIGEST}	Merkle chunks identifying the slot s
$5\ell_{\text{DIGEST}} + c$	ℓ_{DIGEST}	tweak-table hash
multi-message tail (variable, n digests):		
$2\ell_{\text{DIGEST}} + c$	$n\ell_{\text{DIGEST}}$	one digest per inner single-message aggregate (its public-input hash)

The count field holds the number of aggregated public keys for a single-message proof, and the number n of bundled components for a multi-message one.

The last two single-message fields are XMSS-specific. The *Merkle chunks* record the slot s (common to all signatures) split into nibbles. The *tweak-table hash* is the hash (see Section 5.9) of the XMSS *tweaks* (domain separators between every hash of an XMSS) for slot s.

The bytecode claim is one evaluation of the bytecode polynomial over $\mathbb{F}_q$ (Section 7.1.1): an evaluation point together with the resulting value. Its size c is built up as follows:

- the bytecode polynomial has $h_{\text{BYTECODE}}+4$ variables (the $+4 = \lceil \log_2 12 \rceil$ addresses the 12 columns of one instruction), so the point has $h_{\text{BYTECODE}} + 4$ coordinates;
- adding the value gives $h_{\text{BYTECODE}} + 5$ elements of $\mathbb{F}_q$, that is $(h_{\text{BYTECODE}} + 5) \cdot d$ base field elements;
- c rounds this up to a whole number of digests.

A WHIR Optimizations

A.1 Avoid paying for (most of) the zero suffix from polynomial stacking

The polynomial S (in ν variables) committed by WHIR (using an interleaved Reed-Solomon code) is the *stacking* (Section 5.1) of all columns, memory, and access counts, padded with zeros up to the next power of two; its 2^ν evaluations are therefore non-zero only on a prefix of some length L ($2^{\nu-1} < L \leq 2^\nu$). The first WHIR round folds $X_1, \ldots, X_{f_0^{\text{whir}}}$ (where $f_0^{\text{whir}} = 7$ is the initial folding factor), which splits the evaluations (on the boolean hypercube) of S into $2^{f_0^{\text{whir}}} = 128$ contiguous blocks. Each block is Reed-Solomon encoded, and the 128 encoded blocks become the columns of a matrix whose rows are the Merkle leaves.

Because the folded variables are the *most significant* bits, the zeros land in the last blocks: only the first $n_{\text{eff}} = \lceil L/2^{\nu - f_0^{\text{whir}}} \rceil$ blocks are non-zero, so every leaf ends with the same $128 - n_{\text{eff}}$ zeros. Three optimizations follow:

- **FFT:** only the first n_{eff} blocks are FFT encoded (trailing zero blocks encode to zero)
- **Leaf hashing:** we hash each Merkle leaf via an overwrite-sponge, absorbing data right-to-left, so the shared zero suffix yields the same partial state, which we precompute once and reuse for every leaf.
- **Proof size:** opened leaves are sent without the zero suffix (n_{eff} values instead of $2^{f_0^{\text{whir}}}$ per first-round query); the verifier re-appends the zeros before hashing.

A.2 Small-Value Optimization (SVO) and Split-Eq

WHIR opening contains a sumcheck of the form $\sum_{b \in \{0,1\}^\nu} w(b)\, f(b)$, where f is the committed (base-field) polynomial and the weight $w(b) = \sum_i \alpha^i \, \hat{eq}(b, w_i)$ batches the evaluation claims $f(w_i) = y_i$ (we omit 'shift' claims for simplicity, but this is generalizable). In our setting the claims are highly *sparse*: stacking many table columns into a single WHIR instance produces, for each column, a claim w_i whose high-order coordinates are the boolean prefix bits selecting that column, with only the remaining coordinates in $\mathbb{F}_q$.

The *small-value optimization* (SVO), together with the *split-eq* trick (algo. 6 of [14]), speeds up the first ℓ_0 sumcheck rounds (where most of the prover's cost lies) by precomputing accumulators (via the factorization $\hat{eq}(b, w_i) = \hat{eq}(b^{lo}, w_i^{lo}) \cdot \hat{eq}(b^{hi}, w_i^{hi})$). For prover cost to be proportional to the *non-sparse* part of each claim, the boolean (sparse) coordinates must sit at the *opposite end* from the variables folded first.

This conflicts with the zero-suffix optimization of Section A.1, which folds the *most significant* variables first ($X_1, X_2, \ldots$). TODO can we (efficiently) reconcile the 2 optimizations?

References

[1] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Journal on Computing*, vol. 26, no. 5, p. 1484–1509, Oct. 1997. [Online]. Available: http://dx.doi.org/10.1137/S0097539795293172

[2] L. Goldberg, S. Papini, and M. Riabzev, "Cairo – a turing-complete STARK-friendly CPU architecture," Cryptology ePrint Archive, Paper 2021/1063, 2021. [Online]. Available: https://eprint.iacr.org/2021/1063

[3] G. Arnon, A. Chiesa, G. Fenzi, and E. Yogev, "WHIR: Reed–solomon proximity testing with super-fast verification," 2024. [Online]. Available: https://eprint.iacr.org/2024/1586

[4] C. Lund, L. Fortnow, H. Karloff, and N. Nisan, "Algebraic methods for interactive proof systems," *Journal of the ACM*, vol. 39, 04 1999.

[5] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schofnegger, "Poseidon: A new hash function for zero-knowledge proof systems," Cryptology ePrint Archive, Paper 2019/458, 2019. [Online]. Available: https://eprint.iacr.org/2019/458

[6] D. Boneh, B. Lynn, and H. Shacham, "Short signatures from the weil pairing," in *Advances in Cryptology — ASIACRYPT 2001*, C. Boyd, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 514–532.

[7] J. Drake, D. Khovratovich, M. Kudinov, and B. Wagner, "Hash-based multi-signatures for post-quantum ethereum," Cryptology ePrint Archive, Paper 2025/055, 2025. [Online]. Available: https://eprint.iacr.org/2025/055

[8] D. B. Johnson, A. Menezes, and S. A. Vanstone, "The elliptic curve digital signature algorithm (ecdsa)," *International Journal of Information Security*, vol. 1, pp. 36–63, 2001. [Online]. Available: https://api.semanticscholar.org/CorpusID:207063673

[9] D. J. Bernstein, A. Hülsing, S. Kölbl, R. Niederhagen, J. Rijneveld, and P. Schwabe, "The SPHINCS+ signature framework," Cryptology ePrint Archive, Paper 2019/1086, 2019. [Online]. Available: https://eprint.iacr.org/2019/1086

[10] A. Kate, G. M. Zaverucha, and I. Goldberg, "Constant-size commitments to polynomials and their applications," in *Advances in Cryptology - ASIACRYPT 2010*, M. Abe, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 177–194.

[11] S. Papini and U. Haböck, "Improving logarithmic derivative lookups using GKR," Cryptology ePrint Archive, Paper 2023/1284, 2023. [Online]. Available: https://eprint.iacr.org/2023/1284

[12] E. Ben-Sasson, A. Chiesa, and N. Spooner, "Interactive oracle proofs," Cryptology ePrint Archive, Paper 2016/116, 2016. [Online]. Available: https://eprint.iacr.org/2016/116

[13] A. Chiesa and M. Orrù, "A fiat–shamir transformation from duplex sponges," Cryptology ePrint Archive, Paper 2025/536, 2025. [Online]. Available: https://eprint.iacr.org/2025/536

[14] S. Bagad, Q. Dao, Y. Domb, and J. Thaler, "Speeding up sum-check proving," Cryptology ePrint Archive, Paper 2025/1117, 2025. [Online]. Available: https://eprint.iacr.org/2025/1117